

Conceptual scheduling model and optimized release scheduling for agile environments

Ákos Szőke*

Dept. of Measurement and Information Systems, Budapest University of Technology and Economics, H-1521 Pázmány Péter sétány 1/D, I.B. 414 Budapest, Hungary

ARTICLE INFO

Article history:

Received 9 September 2009
Received in revised form 23 November 2010
Accepted 24 January 2011
Available online 1 February 2011

Keywords:

Agile software development
Release planning
Planning metamodel

ABSTRACT

Context: Release scheduling deals with the selection and assignment of deliverable features to a sequence of consecutive product deliveries while several constraints are fulfilled. Although agile software development represents a major approach to software engineering, there is no well-established conceptual definition and sound methodological support of agile release scheduling.

Objective: To propose a solution, we present, (1) a conceptual model for agile scheduling, and (2) a novel multiple knapsack-based optimization model with (3) a branch-and-bound optimization algorithm for agile release scheduling.

Method: To evaluate our model simulations were carried out seven real life and several generated data sets.

Results: The developed algorithm strives to prevent resource overload and resource underload, and mitigates risks of delivery slippage.

Conclusion: The results of the experiment suggest that this approach can provide optimized semi-automatic release schedule generations and more informed and established decisions utilizing what-if-analysis on the fly to tailor the best schedule for the specific project context.

© 2011 Elsevier B.V. All rights reserved.

1. Introduction

Recently, many practitioners have adopted the ideas of agile software development [2]. Aspiring to agile approaches can be explained by recent surveys showed that agile teams are often more successful than traditional ones [3,4]. Several studies pointed out 60% increase in productivity, quality and improved stakeholder satisfaction [4,5], 40% faster time-to-market, and 60% and 40% reduction in pre-, and post-release defect rates [6] comparing to the industry average. The most popular agile methods are Extreme Programming (XP) [7] (58%), Scrum [38] (23%), and Feature Driven Development (FDD) [8] (5%) [9]. Despite the variety of methods, all of them share the core principles of *Agile Manifesto* [2].

In 2008, an Agile Tools survey [10] showed that many developers-focused tools were come out (including JUnit testing, sub versioning, auto builds, etc.) in the last decade, but most companies (>52%) are still using old-fashioned project management tools like MS Project or generic tools like spreadsheets. Surprisingly, 18% of the respondents do not use any tool for project scheduling and tracking at all – although many commercial (such as Rally [11])

and open source (e.g. XPlanner [12]) agile release planning tools are available.

The fourth State of Agile Development survey was conducted in 2009 [1]. The collected data includes information from 88 countries, from 2,570 participants – ranging from project managers, development managers, developers and senior managers. This survey points out that the first and the fourth most important agile techniques from the identified 19 ones are iteration and release planning. Besides these facts, the survey also revealed that the 13 most commonly cited barriers of agile adoption within companies. The three out of the four most important barriers are (1) the lack of upfront planning, (2) the loss of management control, and (3) the lack of predictability. These barriers are closely connected with the present practice of agile planning. These critics underline the importance of providing a more established method for agile planning that lacks of solid theoretical basis currently. In this article, our aim is to diminish these barriers. First, we provide a proposed conceptual model of agile planning that defines a common language. The aim of the model to provide a global and common view for agile planning and to avoid the different interpretations that are used in the field. This model not only helps to identify and define the main notions but with its precise relationships it can also be used in the design of agile planning applications. Presently, the agile release planning, which is different from the ordinary release planning approaches, is unsatisfactory due to lack of solid

* Tel.: +36 30 383 4657; fax: +36 1 288 0518.

E-mail address: aszoke@mit.bme.hu

URL: <http://home.mit.bme.hu/~aszoke/index.htm>

theoretical basis and existence of a number of competing practices and fads. Therefore, as a second aim, we propose a theoretically sound method for agile release planning. The proposal gives the main parameters of the typical agile planning space (such as objectives, constraints) and presents an optimization model that can be realized by optimization tools or by implementing our suggested custom-made algorithm. We believe these results may help to diminish the previously mentioned barriers (1–3) of agile adoption.

1.1. Related work

Projects usually have more candidate requirements than can be realized within the time and resource constraints of the release. Prioritization helps to select the more important requirements from the set of candidates to support release-centered decisions. Basically, there are two different techniques to prioritize requirements. The *absolute* prioritization technique (e.g. MoSCoW rules [13]) determines the importance of requirements without comparing to any other requirement, while the *relative* technique is based on the expressed relative importance among requirements. Relative techniques tend to be more accurate and informative than absolute ones [14]. One relative technique is the \$100-test presented in [15] to assign a fixed amount of units among all requirements by different stakeholders in order to construct relative weighted list of requirements. The Wiegers' method calculates the priority of a requirement by dividing the value of a requirement by the sum of the costs and technical risks associated with its implementation [16]. The requirement's value depends on the business value provided to the customer on a scale from 1 to 9. Karlsson's pair-wise comparison technique [14] is based on the Analytical Hierarchy Process (AHP) [17,18]. In this technique, all requirement pairs are compared according to their importance on the same scale that is employed in AHP: 1 (equal); 3 (moderate difference); 5 (essential difference); 7 (major difference), and 9 (extreme difference). Then these comparisons lead to the understanding of each requirement value regarding to the total value of the requirements. In [19], Karlsson and Ryan introduced an improved technique of the Karlsson's pair-wise comparison technique by using cost and value as high-level factors against which each requirement pair is compared. Finally, in [20], Jung assigned values to each requirement then formulated selection of requirements as binary knapsack problem.

Compared to the extensive research on requirements prioritization only few researches dealt with requirements release planning. Carlshamre formulated release planning in [21] as an Integer Linear Program (ILP) problem, and in [22] as a binary knapsack problem where requirement dependencies were treated as constraints. Ruhe extended the ILP planning approach with stakeholders' opinions [23], and Akker also extended it with some managerial steering mechanism that enabled what-if analysis [24]. Li combined the selection and scheduling to provide requirement selection and on-time-delivery project plan simultaneously [25]. In [27] a case study showed that integration of requirements and planning significantly (>50%) can accelerate UML-based release scheduling, and in [28] a bin-packing-based suboptimal release scheduling solution is presented. In [29] iteration scheduling is formulated as a special form of Resource-constrained Project Scheduling Problem (RCPSP) to provide semi-automatic schedule generation and what-if-analysis for the project manager in agile environments. There are also techniques aimed at release planning, in particular when several stakeholders are involved, such as EVOLVE [31] and Quantitative WinWin [32]. Fenton et al., in [33], utilized Bayes belief network to take uncertainty of effort for resource decisions into consideration. In [34] Chang et al. proposed a method to focus on schedule level minimization but did not examine different productivity level

of developers. In [35] Ngo-The and Ruhe proposed a two-phased optimization approach that combines the planning and resources allocation tasks of releases. This method assumes that each feature is decomposed into a sequence of tasks (e.g. design, implementation and testing) in release planning time. The second phase, which can be used alone in small problems, performs an unfocused search to allocate tasks to developers by formulating the problem as a special case of Job Shop Scheduling Problem and this problem is solved with a genetic algorithm. The optional first phase is used to reduce the search space by formulating the problem as a binary knapsack decision problem. The advantage of the first phase becomes considerable when the feature count is relatively large ($>n * 10$).

Due to the characteristics of agile approaches, agile release planning is significantly different from the previously proposed methods. Agile methods can be seen as a reaction to plan-based (or traditional) methods. Plan-based approaches emphasize the 'rationalized and engineering-based approach' and claim that problems are fully specifiable and predictable solutions exist for every problem. The common release planning methods (e.g. [24,35]) well fit into this claim. Contrary, agile methods generally encourage frequent inspection and adaption where requirements and solutions evolve through collaboration between self-organizing cross-functional teams, they promote business approach that aligns development with customer needs and company goals [36]. Agile methods are a set of development processes – not a single approach to software development – that state that agile development should focus on the following four core values [2]: (1) *Individuals and interactions over processes and tools*; (2) *Working software over comprehensive documentation*; (3) *Customer collaboration over contract negotiation*; and *Responding to change over following a plan*.

As a consequence of the characteristics of agile approaches, agile planning is usually executed in two steps to support the required flexibility [13]: (1) *release planning* – a coarse-grained level plan that considers features (essential abstractions that both customers and developers understand), team resource (human resources in a cumulative way) and several delivery stages (i.e. iterations) (see Section 2.3), and (2) *iteration planning* – a fine-grained level plan that considers technical tasks (working units derived from features), human resources and one delivery stage (i.e. the iteration) (see Section 2.4). So, an agile release is made up of some iterations, and agile planning is carried out in an iterative way by defining: (1) an early coarse-grained level release plan where the major characteristics of the development are known (e.g. deliverable features, the size of the team); and (2) several fine-grained level iteration plans where the detailed expectations and development situation are known (e.g. technical tasks, task dependencies, available developers). After execution of the release the whole process is usually repeated.

The major difference between the common release planning approaches (such as [24,35]) and the proposed agile release planning approach is emphasized in the following. The ordinary approaches assume that deliverable features can be specified in detail at the beginning of the development process, and they are constant during the process. Therefore, these approaches decompose all features into a precise sequence of development and managerial tasks and collect a considerable amount of information (i.e. precise specification, task level effort estimation) up front in the development process. This information is used to realize a release plan in detail (i.e. human resource allocation to all feature realization tasks) for the next months typically.

By contrast, agile processes address both the challenge of an unpredictable world and the alignment with the continuously changing customer needs and company goals. The agile approaches assume that the detailed specification can be gained

with intensive customer collaboration during the process. Therefore, agile methods typically use an adaptive planning approach that includes usually two kinds of plans: a coarse-grained long-time (release), a fine-grained short-time (iteration) plan [13]. As a consequence, during release planning, these methods collect the essential information for the coarse-grained level plan (i.e. feature allocation to stages). These release plans typically last some development months. After release planning, they only collect more detailed information on the features of the next iteration in the given release to realize the fine-grained level plan (i.e. human resource allocation to all tasks of feature realizations). These iteration plans typically last one or two development weeks. These gradually detailed planning approaches require less information collection and planning effort from the project manager during agile release planning. Therefore these plans can be easily adapted to the changes of customer needs and company goals. As a consequence, they provide a more flexible planning approach.

In this paper, we present the core concepts of agile release and iteration planning, but we only propose an agile release planning method by formulating agile release planning as a variation of the multiple knapsack-based optimization model [61]. A complementary agile iteration planning method and a distributed agile release planning extension can be found in [29] and in [30] respectively. Those methods fit well to the presented agile release planning method.

1.2. Problem statement and analysis

The lack of penetration of the modern agile planning tools (see Section 1) can be explained by the weak embedded support of traditionally important project *scheduling functions* such as task assignments and what-if analysis. Their implemented methods provide ‘quick and dirty’ scheduling solutions [11,12]: the team can distribute deliverables among releases and iterations in planning meetings – while all planning constraints are taken into account informally. Typical informally managed planning factors are: (P1) *resource capacities* (resource demands during iterations), (P2) *priorities* (importance of each requirement delivery), (P3) *dependencies* (relations between requirements), (P4) *staged-delivery* (delivery time of iteration timeboxes), and (P5) *maximal value* (to choose the maximal valued one from different plans). Although the principles of agile development (see [2]) rely on communication instead of rigorous planning, this fact can be explained by the lack of easily applicable algorithmic solutions. Informal approaches work well in smaller projects and project manager intuition is necessary, but they are not sufficient in complex decision situations. As the size and complexity increase scheduling becomes a very complex process and advocates tool support [23,26,35]. Moreover, the usual manual approach makes it difficult to construct different plans to perform what-if analysis in order to select the best plan from the potential candidates. As a consequence, optimality of plans (i.e. delivering maximal business value) is heavily based on the manager’s right senses – nevertheless, optimized project plans are crucial issues from the economic considerations of both customer and developer sides. Although, there can be found some general techniques to release planning and scheduling (see Section 1.1), they do not consider the staged-delivery (a release is made up of several iterations) characteristics of agile environments – however, these time factors strongly influence the selection and assignment of requirements to iterations. This staged-delivery approach requires in-depth investigation of feature dependencies that heavily influence on the results and it cannot be carried out without proper tool support.

1.3. Objectives

To provide a systematic approach without omitting the human expert from the process, we propose a formal agile release planning problem description and algorithmic solution which consider the previous factors (P1–5) – while the deficiencies of the informal and intuitive approach are mitigated. Therefore, the main contribution of the paper can be seen as to make release planning process of agile development more objective and more qualified in terms of the results.

Firstly, our intention is to, (1) provide a detailed *conceptual model* of agile planning and scheduling. Then we propose an agile release scheduling method which intends to mitigate previous problems (Section 1.2) by (2) formulating release scheduling as an *multiple knapsack-based optimization model* that considers all the previous factors (P1–5), and (3) providing a solution to this model by a *branch & bound algorithm* to easily produce optimized release scheduling.

1.4. Outline

The rest of the paper is arranged as follows: Section 2 presents elements of agile planning; Section 3 introduces the proposed (1) conceptual model, (2) optimization model and (3) algorithm for agile release scheduling; Section 4 describes experiences; Section 5 discusses our solution; and finally Section 6 concludes the paper.

2. Background

In this section, we introduce the concepts of agile planning to provide the necessary background for the proposed conceptual and optimization model.

2.1. Business value in agile

Business value is a key concept in agile software development [36]. Agile approaches can be considered as a paradigm, a ‘mini-malist software development attitude’ which aim is to carry out those tasks that provide business value for the customer [37]. A business value term is being used in management and economics that includes all forms of value for the company in the long-run. It usually means anything that can be translated to money such as revenue, market share, and stock price – it is generally a multi-dimensional concept.

2.2. Agile features and effort estimation

Common to all software development processes in any project is the need to capture and share knowledge about the requirements and design of the product, the development process, the business domain, and the project status. Contrary to the traditional methods (e.g. waterfall), agile methods advocate ‘just enough’ specifications where the fundamental issue is the communication, not the documentation [38,39]. In agile methods User stories, Features, and Use cases (XP [7], FDD [8], Use cases [40]) are the most popular specification techniques.

Most software products evolve as they go live, therefore there is a need to repair defects that are discovered during operation. Both deliverables (requirements and defect repairs) are usually called *features*. The agreed specification of features not only drives the development, but directs planning of projects and provides the basis for effort estimations [41]. Agile teams focus on ‘good enough’ estimates to try optimizing their estimating efforts. A good estima-

tion approach takes short time, and can be used for planning and monitoring of the project [13].

Requirements effort estimation models are usually based on size metrics: Story points [13], Feature points [8], Use case points [42] and Object-Oriented Function Points [43]. These metrics are abstract units, express the whole size of development and are usually not directly convertible into person hours/days/months. Whereas, defect repairs are usually estimated with person hours/days.

2.3. Agile release planning and scheduling

Release planning and scheduling are activities concerned with the implementation of selected features in the next version of the software. While planning decides *what* needs to be done, scheduling defines *when* to do it. Agile approaches mandate incremental software development approach which aims at delivering smaller releases sequentially instead of delivering the whole system at once. This approach necessitates *staged-delivery release scheduling* process: *selecting* and *assigning* features to consecutive increments. The selection is based on *relative priorities* as a function of business values (Section 2.1) and other factors such as urgencies and risks [31] – depending on the stakeholders' needs. Relative priority values – measured on ratio-scale, which means that priority values are ordered and the ratio between values can be determined [44] – ensure that the features are comparable to support selection. Actually, measurement on ordinal-scale (values can be ordered, but the ratio cannot be determined between elements) is sufficient many cases. But, in our solution, to provide the highest business value to the customer, priority/effort (see ordering of items according to p_i/w_i in Section 3.5.2) quotient is calculated for each deliverable feature, which mandates ratio-scale.

2.4. Iteration planning and scheduling

Once the maximal customer-valued features are selected for the next release the following step is to realize them. In agile approaches, software is rolled out in increments – in *iterations* – to reduce the overall risk of realization [13,9]. Technical tasks are the main concepts of iteration scheduling. These tasks are the fundamental working units accomplished by one developer, and usually require some working hour (*Wh*) realization effort that is estimated by the team. The aim of iteration scheduling is to break down selected features into technical tasks and to assign them to developers [13].

2.5. Date-driven and scope-driven process of release scheduling

Release scheduling usually addresses two kinds of typical customer questions: (1) *Fixed-time* question: 'How much of the features can be delivered by a given date?' and/or (2) *Fixed-scope* question: 'When can the selected (or all) features be delivered?'. The *Fixed-time* question mandates *date-driven*, while *Fixed-scope* one requires *scope-driven* scheduling approach. In fact, the former one can be interpreted as a temporal constrained version of the latter one.

In the following, a high-level algorithmic formulation of the scheduling process is presented. The pseudo-code of the two scheduling approaches is shown in one algorithm to be concise (Algorithm 1). The lines in parentheses and marked with † mean that they are excluded from the scope-driven and included in the date-driven case. While mark ‡ denotes the opposite case. In the program listing lowercase/upercase letters with indices denote vectors/matrices (e.g. $p_i, D_{j,j'}$) and without indices, they mean scalars (e.g. l^R). While bold-faced types show concise (without indices) forms (e.g. $\mathbf{D}$), and Gothic types denote sets (e.g. $\mathcal{R}$).

In the *require* section the preconditions are given. At the beginning of the process, the release constraints – such as required release duration (l^R : in days), available developers ($\mathcal{R}^*$) and their effectiveness factors (e_i : proportion of his/her daily work on this release), and dependencies between features ($D_{j,j'}$) – and business factors – priorities (p_j) and efforts (w_j : in person days) – are defined. The *ensure* section prescribes the post condition on the return value ($\mathbf{X}$): every User story j has to be assigned to maximum one or exactly one iteration k .

Algorithm 1. Date-driven and scope-driven agile release scheduling process

Require: $(l^R \in \mathbb{N})^\dagger$ /*Release duration*/
 $i \in \mathcal{R}^*, e_i \in [0, 1]$ /*Available developers and their effectiveness factors*/
 $j \in \mathcal{W}, p_j \in \mathbb{N}, w_j \in \mathbb{N}, D_{j,j'} \in \{0, 1\}$ /*Priorities, efforts and dependencies*/

Ensure: $\forall j \exists! k (X_{k,j} \in \{0, 1\})^\dagger$ xor $(X_{k,j} = 1)^\ddagger$

- 1: **repeat**
- 2: $i \in \mathcal{R} \subseteq \mathcal{R}^*$ /*Selecting developers*/
- 3: $v^{\mathcal{R}} \leftarrow \sum_i e_i : i \in \mathcal{R}$ /*Calculating the velocity of selected resources*/
- 4: k, l_k^l /*Assessment of iteration count and length of each iteration*/
- 5: $c_k \leftarrow v^{\mathcal{R}} * l_k^l :$ /*Calculating iteration velocities*/
 $k \in \mathcal{J} \left(\bigwedge \sum_k l_k^l \leq l^R \right)^\dagger$
- 6: $\mathbf{X} \leftarrow \text{schedule}(\mathbf{p}, \mathbf{w}, \mathbf{D}, \mathbf{c})$ /*Select and assign features to iterations*/
- 7: **until** $\mathbf{X}$ is satisfying
- 8: **return** $\mathbf{X}$

The main block of the algorithm shows that the release scheduling is an iterative process. Several schedule alternatives – typically due to what-if-analysis – compete with each other until the decision-makers are satisfied (line 1–7). During scheduling, first the developers are selected (line 2) and their velocity (line 3) is calculated. Next, the iteration count of the release, the length of each iteration (in days) is assessed (line 4) and the iteration velocity (i.e. how many features can be delivered by the team in a given iteration) is calculated (line 5). The most complex and critical part of the process is to find a schedule (line 6) where the objective (delivering features with the highest priorities) is maximum while the constraints (i.e. p, w, D, c) are satisfied (line 7).

In this process, the settings of i, k , and l_k^l factors remained informal, their values mainly depend on the development situation and local agile practices. Their typical values are between 4–12 people, 2–4 iteration counts, 1–4 weeks respectively [3,13,9]. It is appropriate to calculate team velocities ($v^{\mathcal{R}}$) from developers' effectiveness factors if historical values are not available, but as soon as they are available, usage of historical values provides more reliable plans [13].

3. Optimized agile release scheduling

In this section, first, we introduce interdependencies between features (Section 3.1) heavily influencing the scheduling process. Secondly, we present a proposed conceptual model (Section 3.2) to provide a global view of agile release and iteration scheduling. Then we point out that agile release scheduling (RS) can be characterized as a specialized multiple knapsack problem, and

we formulate a general optimization model for this problem (Section 3.4) – or in other words, a model for the `schedule` function (Algorithm 1). Finally, we present a solution to this optimization model in the form of a branch & bound algorithm (Section 3.5).

3.1. Dependencies between requirements

The complexity of scheduling arises from the interaction between requirements by *implicit* and *explicit* constraints. While the previous is given by the scarcity of resources, the latter one is emerged from different dependencies between requirements. The main sources of dependencies are identified in [21] and summarized in Table 1. To clarify the meaning of dependencies, the following examples are given in Table 2. (Note, j, j' denote requirements):

Theoretically, there can be $C_n = 6n(n-1)/2 = 3n(n-1)$ number of dependencies (where n denotes the number of requirements, and 6 the number of dependency types). Generally, the number of dependencies is dependent on the case, and in [21] it was found that this number is roughly between $(n, 2n)$.

3.1.1. Mapping requirement dependencies to feature dependencies

Considering release scheduling two important types of dependencies should be identified [23]: (1) *coupling* (C): requirements connected with *AND* dependency should be released jointly because they expect each other to serve a unit of functionality, (2) *precedence* (P): requirements connected with *REQUIRES*, *CVALUE*, *ICOST* or *TEMPORAL* dependencies should be released in given temporal order (i.e. j' before j). Dependency *XOR* only influences preliminary requirement selection during the requirement's spec-

Table 3

Mapping requirement dependencies to feature dependencies.

Requirement dependency group	Type	Influence on selection (see [21])	Influence on RS	Feature dependency
Functionality-related dependency	<i>AND</i>	✓	✓	C
	<i>REQUIRES</i>	✓	✓	P
	<i>XOR</i>	✓	×	–
Value-related dependency	<i>CVALUE</i>	✓	✓	P
	<i>ICOST</i>	✓	✓	P
Time-related dependency	<i>TEMPORAL</i>	×	✓	P

ification phase and does not influence on release scheduling. Table 3 summarizes the previous mappings.

Besides requirements' implementation, defect repairs should also be ordered during scheduling thus we interpret these two dependencies (i.e. *coupling* and *precedence*) and call them *feature dependencies* (cf. Section 2.3).

3.1.2. Constructing singular feature dependencies

Feature dependencies can be modeled as binary relations. Precedence dependency can be defined as partial order relation [48] (usually written as ' $\leq$ ') and it possesses the Eqs. (1a), (1c) and (1d) properties from the following:

$$\text{If } j \in \mathcal{W}, \text{ then } j \sim j \text{ (reflexive)} \quad (1a)$$

$$\text{If } j \sim j', \text{ then } j' \sim j \text{ (symmetric)} \quad (1b)$$

$$\text{If } j' \sim j \text{ and } j \neq j', \text{ then } \neg(j \sim j') \text{ (anti-symmetric)} \quad (1c)$$

$$\text{If } j' \sim j \text{ and } j'' \sim j', \text{ then } j'' \sim j \text{ (transitive)} \quad (1d)$$

Analogically, *coupling* can be interpreted as equivalence relation [48] (usually written as ' $=$ ') so it splits the set of features up (i.e. partition) into strict subsets which group jointly releasable items. Therefore *coupling* possesses the Eqs. (1a), (1b) and (1d) properties from the Eq. (1).

There can be not just singular, but multiple dependencies between two features. It is obvious that similar multiple dependencies can be expressed by one singular relation since the properties of the dependencies are the same. Although, multiple different dependencies may result in conflicts in dependencies – though release scheduling deems necessary the unanimous instructions to the implementation order. Problematic cases are those where (1) anti-symmetric properties are in opposite directions (i.e. $j' \leq j$ and $j \leq j'$) and/or (2) both anti-symmetry and symmetry properties are expected (e.g. $j' \leq j$ and $j' = j$). Nevertheless, if we consider the fact that these relations express temporal dependencies on the time line (since the aim is to order items), then for both previous cases the proper singular relation is ' $=$ '.

3.2. Conceptual model of agile release and iteration scheduling

In order to formulate the release scheduling optimization model, first, we have to identify the main concepts of agile scheduling. Next, we do not only present release but iteration scheduling concepts to provide a global view of agile scheduling. Thus, these concepts not only help to identify the objects and the subject of the optimization model but with the precise relationships it can also be used as database schema definition for agile release and iteration scheduling applications.

Our proposed conceptual model of agile release and iteration scheduling (Section 2) is visualized with UML notation in Fig. 1 and presented in Table 4 [13,40]. Generally, scheduling mandates defining *who* will realize *what* and *when*. *Team*, *Feature* and *Release*

Table 1
Type of dependencies between requirements.

Dependency name	Definition
<i>AND</i>	j and j' jointly serve a unit of functionality
<i>REQUIRES</i>	j builds on functionality j' to function
<i>XOR</i>	a function can be served by either j or j'
<i>CVALUE</i>	j' influences the customer value of j , so rational to realize j' earlier
<i>ICOST</i>	j' influences the implementation cost of j , so rational to realize j' earlier
<i>TEMPORAL</i>	technological/organizational constraints between j and j' , so it is rational to realize j' earlier

Table 2
Examples of dependencies.

Dependency name	Example
<i>AND</i>	A customer report functionality needs a report visualization component (j) (e.g. Crystal Reports [45]) and a data query (j') to function
<i>REQUIRES</i>	An email alert function (j) <i>requires</i> that the given users to be stored with their appropriate attributes (j') in the system
<i>XOR</i>	A customer report visualization function may be based on Crystal Reports component (j) <i>xor</i> on an in-house solution (j')
<i>CVALUE</i>	A well-organized user interface (e.g. with determined form fill in sequence) (j') usually adds value to the user experience and diminishes the need of detailed user documentation (j)
<i>ICOST</i>	Implementing a configurable report functionality (j') often adds extra cost to the implementation, but it may ease the development of data queries (j)
<i>TEMPORAL</i>	Extending the customer module (j') should be implemented before the development of the customer report (j) since the previous may effect on the report contents

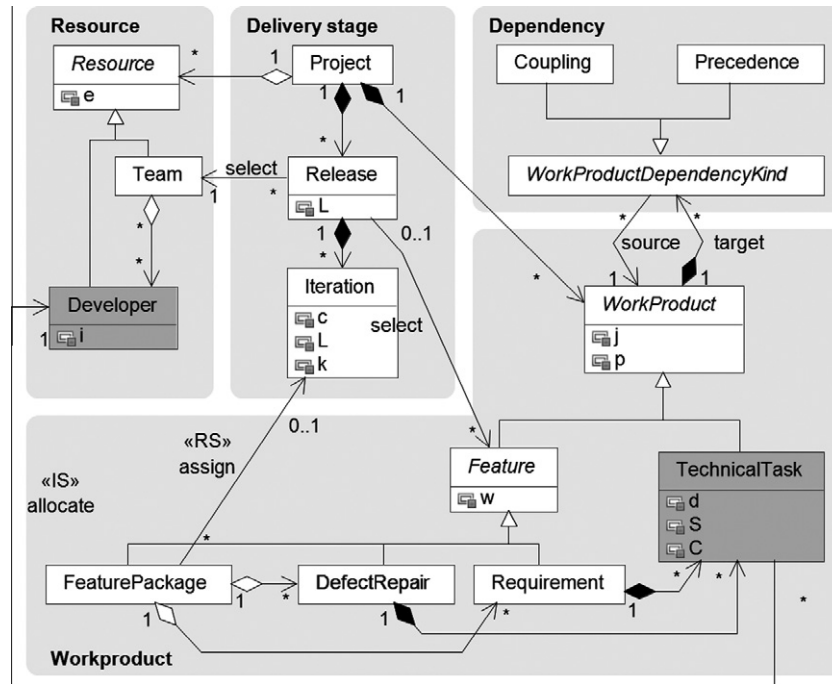


Fig. 1. Conceptual model of agile release and iteration scheduling.

Table 4
Concepts of agile release and iteration planning.

Concept	Description	Notation (Section 2.5)
Project	Is a planned endeavor, usually with specific <i>workproducts</i> that are rolled out in several deliverable stages i.e. <i>releases</i> by some <i>resources</i>	
Release	Produces (usually external) <i>selected</i> deliverable <i>features</i> for the customer by a <i>selected team</i> , and it usually contains 1–4 <i>iterations</i> . In case of date-driven planning the length (or deadline) of release is defined (l^R) in working months/weeks/days	
Iteration	Is a development timebox in which intermediate deliverables i.e. <i>features</i> are <i>selected</i> and <i>assigned</i> to the iteration. It is characterized by available developers ($\sum_i$) and iteration length $L [l_k^I]$ – often expressed by iteration capacity (or velocity) $c [c_k]$: how many features can be delivered by the <i>team</i> in a given iteration index k within the release	k
Resource	Is an abstract concept of human manpower selected for a given <i>project</i> . A resource can be a <i>developer</i> or a <i>team</i> . Each resource is characterized its effectiveness factor ($e [e_i]$) that gives how effectively can take part in the project beside its other activities (e.g. other projects, support tasks). In case of developers its value is between $[0,1]$, while in case of team the effectiveness factors of individual members are aggregated (i.e. $e = \sum_i e_i$)	
Developer	Is the unit of human manpower. In iteration planning, developers (its index is i) are <i>allocated</i> to low level <i>workproducts</i> i.e. <i>technical tasks</i>	i
Team	Is a group of developers <i>selected</i> to the realization of a <i>release</i> from available <i>developers</i> . During release planning, high level <i>workproducts</i> i.e. <i>features</i> are realized by the <i>team</i>	
Workproduct	Abstract concept of deliverables. At the release planning level, deliverables are <i>features</i> , while at the iteration planning level they are <i>technical tasks</i> . Every <i>workproduct</i> (its index is j) is characterized with its value for the customer which is denoted by priority ($p [p_j]$)	j
Feature	Is an abstract concept of deliverable. It is selected for a given <i>release</i> and they can be classified into three kinds of set of elements: <i>feature packages</i> , <i>requirements</i> , and <i>defect repairs</i> . Its realization usually needs several working days (Wd) manpower that is estimated by developers or some methods ($w [w_j]$)	
Requirement	Is a deliverable that is value for the customer. A requirement can be new or changed (including functional and non-functional ones). In most cases requirements mandate several realization steps that may include cooperation of some developers	
Defect repair	Deliverable that fixes defects in former product variants, and in some cases it may include cooperation of some developers	
Feature package	Holds together some features (expressed by <i>Coupling</i> relations) that must be delivered together to be valued for the customer. Its resource demand is aggregated value of its parts ($\sum w_j$) (see Section 3.5.1)	
Technical task	Fundamental working unit accomplished by one developer. Proper coordination requires individually realizable working units thus each <i>requirement</i> and <i>defect repairs</i> should be broken down into several technical tasks. They usually require some working hour (Wh) manpower that is estimated by developers and denoted by duration (d). Additionally, every technical task is characterized with its start/completion dates (S and C) to precise scheduling	
Work product dependency	Abstract concept of dependencies between <i>workproducts</i> . A dependency can be a <i>precedence</i> or a <i>coupling</i>	
Precedence	Realization precedences between <i>requirements</i> , <i>defect repairs</i> and <i>technical tasks</i>	$P_{f,j}$
Coupling	Realization precedence between <i>requirements</i> and <i>defect repairs</i>	$C_{f,j}$

concepts answer to these questions in agile release scheduling (cf. Sections 2.2, 2.3). In this case, a *Feature* can be a *Requirement* or a *DefectRepair*; and a *Release* consists of several *Iterations*. Whereas, *Developer*, *TechnicalTask* and *Iteration* concepts answer to the previous questions in agile iteration scheduling (Section 2.4). Due to the structural similarity of release and iteration scheduling, these concepts are expressed with *Resource* and *Workproduct* abstract concepts. Additionally, schedule constraint must also be defined: resource constraints can be expressed by iteration velocity or the number of developers (Section 2.5), temporal constraints can be asserted as attributes (e.g. deadlines – Section 2.5) or *Precedence* and *Coupling* dependencies (Section 3.1.1).

In Fig. 1, shaded objects (like *Developer* and *TechnicalTask*) pertain to iteration scheduling only. Objects in italics point out abstract objects (cannot be instantiated), and the lower compartments of entities with double compartment give the list of attributes of each object.

3.3. Mapping to multiple knapsack problem

In this section, we specify agile release scheduling as a combinatorial optimization problem. In such a problem, we have to find the maximum valued solution from a finite but very large number of solutions. Generally, the 0-1 *knapsack problem* (BKP) instance is specified by a knapsack capacity and a set of items where each item has profit and weight. The objective is to fill up the knapsack by selecting from among possible items those which give maximum profit while the total size of items does not exceed the knapsack capacity [46,47]. An important generalization of the 0-1 knapsack problem is the 0-1 *multiple knapsack problem* (BMKP) which arises when more than one knapsack is given with possible different capacities. The analogy between release scheduling and multiple knapsack problem can be explained as follows.

The team's iteration velocity (cf. v^* in Section 2.5) in an iteration stands for a knapsack capacity, while a feature's resource demand (cf. w_j in Section 2.5) represents an item weight. We can view each iteration (cf. k in Section 2.5) within a release as a knapsack into which we can select and assign different deliverable features. Without loss of generality, we can ensure that the resource demand of each feature is less than team's iteration velocity (i.e. knapsack capacity). We apply priority instead of profit to express values of features.

The two release scheduling question types (cf. Section 2.5) can be formed as:

- **Fixed-time:** We have a fixed release date to which we determine an iteration count using the iteration velocity. In this case, the goal is to maximize the delivered value in the determined iterations (knapsacks).
- **Fixed-scope:** We have a fixed scope to which we determine how many iterations are needed. In this case, the goal is to minimize the number of iterations (knapsacks) used.

It is obvious that the former one is similar to BMKP. Although, the latter one should be formulated with flexible packing approach: we are packing features until all selected requirements are assigned to iterations (not fixed iteration count). At the point of scheduling we assume that the number of candidate features is larger than what can be developed with the available resources. Therefore, an important common characteristic of both approaches is that – when we pack deliverables into iterations – we usually select the highest valued deliverables first in order to maximize the delivered values. It has an important practical advantage in the Fixed-scope case: it makes that possible to mitigate the emerging problems on the most valued deliverables early.

We extend the ordinary BMK problem and interpretation with the following elements to provide further computational capabilities for wide-ranging scheduling situations (cf. Section 1.2: **P1–5**): (1) flexible packing approach, (2) dependencies between features (cf. Section 3.1). From now on we call this extended problem as agile release scheduling problem (ARSP).

3.4. Formulating ARS optimization model

Let us given a set of deliverable features $j (j \in \mathcal{W} : |\mathcal{W}| = n)$ with resource demands w_j and iterations $k (k \in \mathcal{J} : |\mathcal{J}| = o)$ with different iteration velocities c_k within a release. In case of Fixed-scope scheduling $o \leq n$ (considering n features the iteration count cannot be more than n), and in case of Fixed-time scheduling $o \triangleq \lceil FT / \sum_k l_k^i \rceil \leq n$, where FT is the Fixed-time duration of the release and l_k^i is the length of iteration k . Let assign each feature into one iteration so that the total required effort in iteration k does not exceed c_k , and the number of iteration used as a minimum – while both precedence relations (matrix) $P_{jj'} \in \{0, 1\}$ (where $P_{jj'} = 1$ if j precedes j' , otherwise $P_{jj'} = 0$) and coupling relations (matrix) $C_{jj'} \in \{0, 1\}$ (where $C_{jj'} = 1$ if j is coupled with j' , otherwise $C_{jj'} = 0$) hold. A possible mathematical formulation can be found in (2) (see definitions of p_j , w_j in Section 2.5). The equation marked with $\dagger$ means the date-driven (some tasks may be omitted due to lack of resource) case while $\ddagger$ denotes the scope-driven (every task must be done in some iteration) case.

$$\text{Maximize } z = \sum_{k=1}^m \sum_{j=1}^n p_j x_{kj} \quad (2a)$$

$$\text{subject to } \sum_{j=1}^n w_j x_{kj} \leq c_k y_k : \quad k = 1, \dots, m \quad (2b)$$

$$k' - k \geq P_{jj'} : \quad x_{k'j'} = x_{kj} = 1 \quad (2c)$$

$$\lfloor 1 / (|k' - k| + 1) \rfloor = C_{jj'} : \quad x_{k'j'} = x_{kj} = 1 \quad (2d)$$

$$\left(\sum_{k=1}^m x_{kj} \leq 1 \right)^\dagger \text{ xor } \left(\sum_{k=1}^m x_{kj} = 1 \right)^\ddagger : j = 1 \dots n, \quad \forall j \exists! k \quad (2e)$$

where $y_k = 0$ or 1, and $x_{kj} = 0$ or 1, and

$$x_{kj} = \begin{cases} 1 & \text{if } j \text{ is assigned to iteration } k \\ 0 & \text{otherwise} \end{cases} \quad (3a)$$

$$y_k = \begin{cases} 1 & \text{if iteration } k \text{ is used} \\ 0 & \text{otherwise} \end{cases} \quad (3b)$$

The equations denote: (2a) deliverable value maximization of release (i.e. summing up values of delivered features p_j in each iteration k), (2b) resource constraints (i.e. resource demands of delivered features w_j in iterations cannot be greater than the total effort in different iterations c_k), (2c) temporal constraints (i.e. if j precedes j' ($P_{jj'} = 1$) then iteration index k' must be greater than or equal to k), (2d) coupling constraints (i.e. if j is coupled with j' ($C_{jj'} = 1$) then iteration index k' must be equal to k), and finally (2e) item j can be assigned to exactly one (scope-driven case) or maximum one (date-driven case) iteration depending on the goal of the decision maker (cf. Section 2.5).

We will suppose, as is usual, that the efforts w_j are positive integers. Without loss of generality, we will also assume that

$$c_k \text{ is a positive integer} \quad (4a)$$

$$w_j \leq c_k \text{ for } \forall k, j \quad (4b)$$

If the assumption (4a) is violated, c_k can be replaced by $\lfloor c_k \rfloor$. If an item violates the assumption (4b), then the instance is treated as trivially infeasible. For the sake of simplicity, we will also assume that, in any feasible solution, the lowest indexed iterations are used, i.e. $y_k \geq y_{k+1}$ for $k = 1, 2, \dots, n - 1$.

3.5. Solving the ARS optimization problem

For the previously formulated optimization model, we developed a *multiple knapsack scheduling algorithm* (Algorithm 2). It is a branch & bound algorithm, which iteratively selects and schedules an item (*feature*) into an *iteration*.

3.5.1. Constructing feature packages

The structural analysis of the ARSP model leads to the decomposition of the release schedule into a set $\mathcal{W}$ of features, a set P of precedence and a set C of coupling relations. These elements can be represented as a *feature graph* $G_F = \langle \mathcal{W}; P, C \rangle$, where the precedence and the coupling relations are drawn as directed and non-directed edges respectively. Therefore, G_F is a multigraph as the same two vertices may have two edges. One can notice that $G_F = G_P \cup G_C = \langle \mathcal{W}; P \rangle \cup \langle \mathcal{W}; C \rangle = \langle \mathcal{W}; P \cup C \rangle$, so G_P and G_C are spanning subgraphs of G_F [48].

According to the coupling definition (cf. Section 3.1) coupled features should be delivered together. Therefore, deliverable items may be interpreted as *feature packages*, where coupled features are joined. As a consequence, a graph G_F has to be transformed into a *feature package graph* G_{FP} where precedences are preserved. This G_{FP} can be constructed in the following way.

For each $j \in \mathcal{W}$ we define an *equivalence class* containing j – denoted by $[j]$, where j refers to the equivalence class representative element – to be the set of those elements which are accessible from j through coupling edges. So, each item j of $\mathcal{W}$ belongs to some equivalence class ($j \in [j]$) and the union of all equivalence classes is $\mathcal{W}$. Then we can say that the equivalence class (coupling relation) *partitions* $\mathcal{W}$. As a consequence, we can define coupled items as $[j]$ and the set of them as $\mathcal{W}^C \triangleq \bigcup [j]$. As for precedence relation, we have to construct a new precedence relation – namely P^C – on $\mathcal{W}^C$ considering P on $\mathcal{W}$. As for construction, we have to think of the following two consequences of the previous coupling: (1) some precedences are between items that are in the same partition $[j]$, and (2) some ones are in different partitions – let us say $[j]$ and $[j']$. The former one produce self (not valid) precedences (i.e. $P_{j'j} : j' \in [j]$) in $\mathcal{W}^C$ therefore we have to leave them out. The latter one, however, can produce – possible multiple – valid precedences (i.e. $P_{j'j} : j' \in [j'], j \in [j]$). In case of multiple precedence we need to preserve just one of them.

Fig. 2 shows a feature package construction example using post-mortem analysis results of a real life development at Multilogic [51]. In this figure, precedence is denoted with directed edge notation and coupling is drawn in a cluster. For instance, on the left, there is a coupling relation between nodes indexed with 17, 18, and there is a precedence relation between nodes 6 and 11. In this case, after graph transformation (i.e. $G_F \rightarrow G_{FP}$), 13 dependent

nodes are produced from the 19 – as the coupled nodes are merged. (Notice, (1) transformation reduces the indices in the resulted graph so nodes 1, 5 became 1, (2) nodes 17, 18 and 19–21 on the left have no precedence after transformation, so they are – 16 and 17 respectively – not shown on the right.)

3.5.2. Algorithm for feature package scheduling

Nowadays, branch & bound algorithms are the most common way to effectively find the optimal solution of knapsack problems. In our case, a BKP branch & bound algorithm was embedded into an iterative frame to solve the previously described ARS problem. Up to now several algorithms have been proposed to solve the BKP, although the Horowitz–Sahni algorithm is one of the most effective [47]. We extended this algorithm with (1) multiple knapsack capabilities, (2) flexible packing approach (cf. Section 3.3), and (3) precedences between items (Section 3.1). The first extension is realized by embedding the Horowitz–Sahni algorithm into an iterative frame, while the second and third ones are carried out by interpreting them as additional constraints.

In the program listing letters are interpreted as in Algorithm 1. The algorithm assumes that the items are ordered according to decreasing values of the priorities per resource demands, i.e. $p_1/w_1 \geq p_2/w_2 \geq \dots \geq p_n/w_n$. The course of the proposed algorithm is outlined as follows. It solves the 0-1 multiple knapsack problem as the solution of several 0-1 knapsack subproblems (see first while in the Algorithm 2). In each subproblem solving, first the schedulable (not scheduled and not preceded) items are selected. Two kinds of moves produce the optimal solution of each subproblem: *forward move* inserts the largest possible set of new consecutive items into the current solution; *backward move* removes the last inserted item from the current solution. Whenever a forward move performed, an upperbound of the current solution is computed and compared to the best solution so far to check whether a new forward move may lead to a better solution otherwise backtracking performs. The algorithm stops if backtracking is not possible.

An important feature of the algorithm is that it deals with precedences in a *soft* way: if not every item is packed and room remains in a given iteration, it tries to put consecutive items (that are *directly* preceded by packed items) into the given iteration. Therefore, for example, even if there is a precedence relation between two items, both may be put to the given iteration.

In the *Require* section the preconditions are given. Each w_j is the resource demand for item j . Precedences between items are represented by a matrix where $P_{jj} = 1$ means that item j' precedes item j , otherwise $P_{jj} = 0$. Both conditions $P_{jj} = 0$ (no loop) and P is directed acyclic graph (DAG) ensures that temporal constraints are not trivially unsatisfiable. Additionally, priorities p_j , and iteration velocities c_k are given. The date-driven case can be controlled with the length of c_k (cf. Section 2.5). The *Ensure* section prescribes the two kind of postconditions on the return value ($\mathbf{X}$) depends on scheduling question (cf. Section 2.5, (2e)).

During scheduling steps, first the initial values are set (lines 1–4). The assignment matrix $\mathbf{X}$ contains the feature package assign-

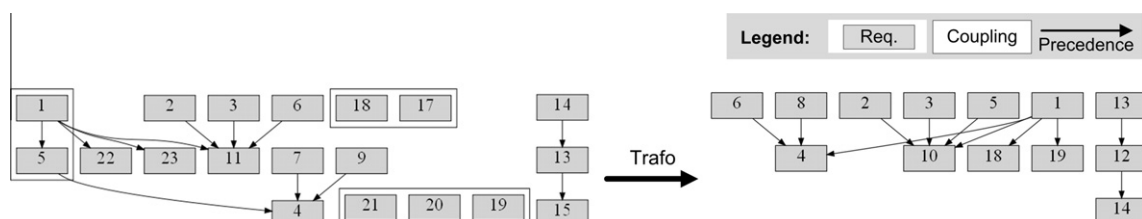


Fig. 2. Illustration of feature package construction.

ments to iterations – initially it is set to a zero matrix (*items* $\times$ *iteration count*) (line 1). The algorithm uses a *ready list* (**rlist**) and a *scheduled list* (**slist**) to keep track of schedulable and scheduled items – these lists are set to empty sets (line 2). Finally, the iteration index (*idx*) is initialized (line 3) and the signal of remained velocity usage for soft precedence handling is initialized (line 4).

The first *while* iterates as long as there is any schedulable item and iteration (line 5). The algorithm selects unassigned items (**ns**) (line 6) then constructs the actual ready list (**rlist**) from which the algorithm can choose in the current control step without violating

any precedence constraint (line 7). If the ready list does not contain any item the schedule is infeasible (line 9).

The second *while* iterates until the actual iteration is filled optimally (line 21). Before the cycle, the current list of priorities (**pn**) and resource demands (**wn**) are determined using the ready list (line 11). Then the current and the best solutions (assignment vector in the given iteration) and their values are initialized (lines 13 and 14). If all the items are not packed to the given iteration then the current residual iteration velocity (*cn*) is equal to the velocity of an iteration (line 18) else it is equal to the remained velocity of the iteration (line 16) – see it later (line 48).

Algorithm 2. mksched algorithm (2/1)

Require

$w_j \in \mathcal{W}$

$P_{jj'} \in 0, 1 \wedge P_{jj} = 0 \wedge \mathbf{P}$ is DAG

$p_j \in \mathbb{N}, c_k \in \mathbb{N}$

Ensure $\forall j \exists ! k (X_{k,j} \in \{0,1\})^\dagger \text{ xor } (X_{k,j} = 1)^\ddagger$

1: $\mathbf{X} \leftarrow [0]_{\text{length}(\mathbf{w}), \text{length}(\mathbf{c})}$

2: **rlist** $\leftarrow \emptyset$, **slist** $\leftarrow \emptyset$

3: *idx* $\leftarrow 1$

4: *remcap* $\leftarrow \text{false}$

5: **while** (*findNotSchedItems* (**slist**) $\neq \emptyset$) $\wedge$ (*length*(**c**) $\geq$ *idx*) **do**

6: **ns** $\leftarrow \text{findNotSchedItems}$ (**slist**)

7: **rlist** $\leftarrow \text{findNotPrecltems}$ (**ns**, **P**)

8: **if** **rlist** = $\emptyset$ **then**

9: **print** “Infeasible schedule!”, **return** $\emptyset$

10: **end if**

11: **pn** $\leftarrow \mathbf{p}(\mathbf{rlist})$, **wn** $\leftarrow \mathbf{w}(\mathbf{rlist})$

12: *n* $\leftarrow \text{length}$ (**rlist**)

13: **x** $\leftarrow [0]_n$, **xn** $\leftarrow [0]_n$

14: *z* $\leftarrow 0$, *zn* $\leftarrow 0$

15: **if** *remcap* **then**

16: *cn* $\leftarrow \text{cnrem}$

17: **else**

18: *cn* $\leftarrow c(\text{idx})$

19: **end if**

20: *backtrack* $\leftarrow \text{false}$, *opt* $\leftarrow \text{false}$, *j* $\leftarrow 0$

21: **while** $\neg \text{opt}$ **do**

22: **while** *j* $\leq n \wedge \text{backtrack} = \text{false}$ **do**

23: *u* $\leftarrow \text{computeUBound}(j)$

24: **if** *z* < (*zn* + *u*) **then**

25: **while** *wn*(*j*) $\leq \text{cn}$ **do**

26: *cn* $\leftarrow \text{cn} - \text{wn}(j)$, *zn* $\leftarrow \text{zn} + \text{wn}(j)$, *xn*(*j*) $\leftarrow 1$

27: *j* $\leftarrow j + 1$

28: **end while**

29: **if** *j* $\leq n$ **then**

30: *xn*(*j*) $\leftarrow 0$

31: *j* $\leftarrow j + 1$

32: **end if**

33: **else**

34: *backtrack* $\leftarrow \text{true}$

35: **end if**

36: **end while**

37: **if** *zn* > *z*

38: **x** $\leftarrow \mathbf{xn}$

39: **end if**

40: *xnidx* $\leftarrow \text{findAssigned}(\mathbf{xn})$

/*weights of each feature *j**/

/*precedences*/

/*priority values and iteration velocities*/

/*assignment matrix initialization*/

/*‘ready list’ and ‘scheduled list’ initialization*/

/*iteration index initialization*/

/*remained velocity is not utilized*/

/*find not scheduled items*/

/*construct ready list*/

/*current priorities and resource demands */

/*items in the ‘ready list’*/

/*best and current assignment vector*/

/*best and current values*/

/*let current residual velocity equal to remained velocity*/

/*let current residual velocity equal to iteration velocity*/

/*backtracking, optimum, search index*/

/*– cycle of subproblems –*/

/*– cycle of forward move –*/

/*computing upper bound from *j**/

/*forward move may lead to better solution*/

/*insert the largest set of consecutive items*/

/*take away it from the current solution*/

/*forward move does not lead to better solution*/

/*forward move produced better solution*/

/*update the current solution as the best one*/

Algorithm 3. *mksched* algorithm (2/2)

```

41:  k = 0
42:  if isEmpty(xnidx) then                                     /*backtracking is no more possible*/
43:    opt ← true
44:    X(rlist(findAssigned(x))) ← 1                             /*store scheduled items*/
45:    slist(rlist(findAssigned(x))) ← 1                         /*update scheduled-list*/
46:    if isEmpty(findNotSchedItems(slist)) then                 /*not scheduled items*/
47:      minnsi ← minWeightOfNotSchedItems(slist)
48:      cnrem ← cn – sum(findWeightOfAssigned(x))              /*remained vel.*/
49:      if cnrem < minnsi then
50:        remcap ← false
51:        idx ← idx + 1                                           /*increment iteration*/
52:      else
53:        remcap ← true                                           /*remained velocity utilization*/
54:      end if
55:    else
56:      idx ← idx + 1                                           /*increment iteration*/
57:    end if
58:    P{1,...,n}, j = 0                                           /*delete scheduled items*/
59:  else
60:    k ← max(xnidx)                                           /*find the last inserted item*/
61:  end if
62:  if !opt ∧ (backtrack ∨ j > n) then                         /*- backtracking -*/
63:    cn ← cn + wn(k), zn ← zn – pn(k), xn(k) ← 0
64:    j ← k + 1
65:    backtrack = false
66:  end if
67: end while
68: end while
69: return X

```

The third *while* (line 22) represents forward moves and iterates while the search index is not out of scope ($j > n$) and backtrack is not necessary. Whenever a forward move performed, an upper-bound from index j relating to the current solution is computed and compared to the best solution so far to check whether a new forward move may lead to a better solution (lines 23 and 24) otherwise backtracking performs (line 34). If a forward move could lead to a better solution than the algorithm inserts the largest set of consecutive items into the solution (lines 25–28). If the resource demands of the last item, where $j \leq n$, exceeds the remained iteration velocity then it is set to 0 in the current solution (lines 29–32).

If forward move produced a better current solution than the best solution so far, we update the best solution (lines 37–39). If backtracking is no more possible (line 42) then the optimal solution of the subproblem was found. As a consequence, it stores the assigned items (line 44), actualizes the scheduled-list (line 45), and deletes the no longer valid precedences (lines 58). Although, if there are not scheduled items (line 46), and if the remained velocity of the iteration – which is calculated by initial velocity minus weights of scheduled items (line 48) – is greater than the minimum weight of schedulable items (lines 47, 49) then remained velocity utilization is set (line 53) else it is annulled and the iteration count is incremented (lines 50 and 51). If the current solution is not empty, it calculates the last inserted item's index (line 60) which is used in backtracking (lines 62–66).

After termination, *X* contains the item assignments to iterations, where the number of nonzero columns denotes the packed iterations (cf. (2a)):

$$z \leftarrow \text{length} \left(\text{nonZeros} \left(\sum_{j=1}^n w_j x_{kj} \right) \right) \quad (5)$$

The presented branch & bound approach produces global optimal solutions. In the course of computation it builds a tree, where each node corresponds to the inclusion or omission of an item. As we have n items and o iterations, there will be 2^n nodes and o trees so the time complexity is $O(o * n^2)$. Since several packing combinations are not feasible, the tree has less nodes in practice so it provides sufficient results for practical applications [47]. As previously stated, it assumes that the items are sorted, if it is not the case, they can be re-indexed in $O(n * \log n)$ time through an efficient sorting algorithm (e.g. Quicksort).

We need to underline that the backtracking capability of the presented algorithm ensures finding the global optimum. The precedences determine the selectable items set (see the ready list in line 7) at a given time, therefore the selection of a given item from this set may lead to suboptimal results as it determines the further search space. So, in order to correct the previous selection which did not lead to optimal solution, we have to backtrack and rethink our decision by select an other one. This process is realized in this algorithm as follows: if forward move does not lead to a better solution (line 34), it finds the last inserted item (line 60), and this item is taken out from the current solution (line 63), then it continues to find better solution with forward move (line 22). During this process the precedences of the given solution are maintained according to the scheduled items and the precedences (lines 6 and 7).

As previously stated, the presented algorithm is extended the Horowitz-Sahni algorithm [47] in many ways while the core branch & bound search idea of the original algorithm is unchanged. Namely, we extended this algorithm with (1) multiple knapsack capabilities, (2) flexible packing approach (cf. Section 3.3), and (3) precedences between items (Section 3.1). The first extension is realized by dealing with different knapsack capacities (i.e. itera-

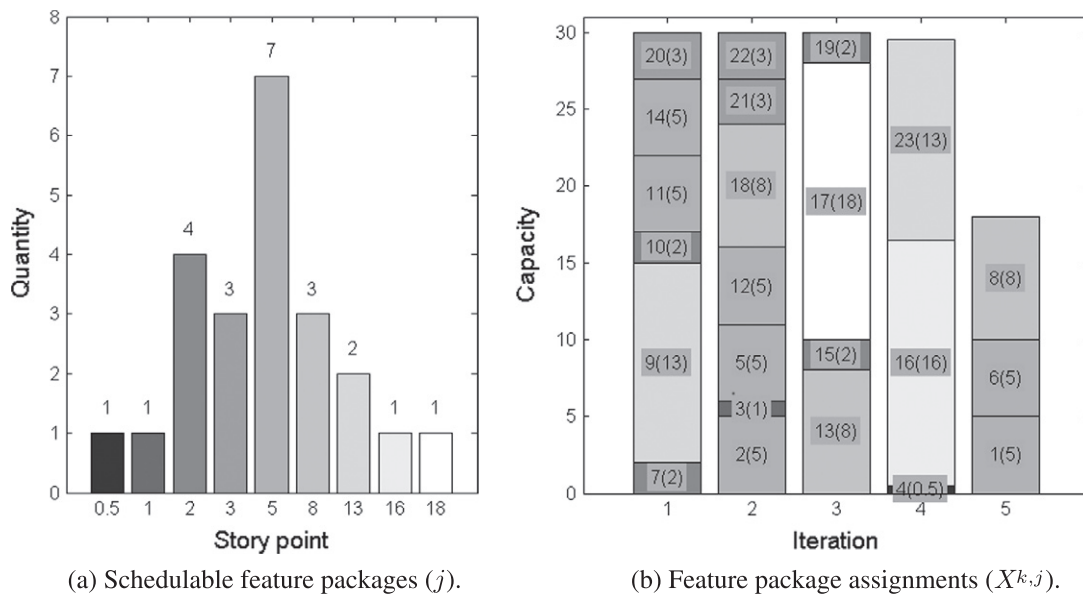


Fig. 3. Release scheduling using the *mksched* algorithm (Algorithm 2).

tion velocities c_k) and embedding the Horowitz–Sahni algorithm into an iterative frame (the external loop, lines 5–68) in order to support multi-iteration scheduling within a release. The second extension is simply carried out by a well-formulated conjunction in the condition of the external loop (line 5). The first part of the conjunction ensures the scope-driven, the second part ensures the date-driven scheduling. Finally, the third extension is realized by introducing the ready list (**rlist**) and the scheduled-list (**slist**) to keep track of schedulable and scheduled items considering the precedence constraints between features (P) during the course of the algorithm.

Fig. 3 illustrates the scheduling concept continuing the previous example (see Section 3.5.1). It shows the post-mortem release scheduling result based on a real life development situation using the Algorithm 2. Fig. 3a shows the histogram of schedulable feature packages. The x-axis enumerates the estimated working days, while y-axis shows how many feature packages fall into these categories. Fig. 3b depicts the scheduling results produced by *mksched* algorithm in stacked bar chart form (where labels denote the indices of feature packages and values within brackets show the size of the packages): the previous feature packages are assigned to five iterations (x-axis) with capacities 30, 30, 30, 29.5, and 18 (y-axis). Bar colors in Fig. 3a point out how feature packages are distributed in Fig. 3b, and in each bar, the values denote indices and the working days (in brackets) of the given feature package.

4. Experimentations

To obtain a proof-of-concept, we implemented a prototype as a scheduling toolbox in Matlab [50]. Seven past release data sets – extracted from the backlog of IRIS application developed by Multilogic Ltd [51] – were compared against the results of simulations applying the same inputs [49]. Additionally, we also carried out simulations on generated hypothetical data sets to get an insight into the performance of the presented algorithm on larger problems.

4.1. Context and methodology

IRIS is a client risk management system (approx. 2 million SLOC) for credit institutions for analyzing the non-payment risk

of clients. It has been a continual evolution since its first release in the middle of 1990s. The system was written in Visual Basic and C# the applied methodology was a custom agile process. The release scheduling process was made up of the following steps. First, the project manager used intuitive rules for selecting features from the backlog into a release. Then the team estimated on every feature and determined the number and the length of iterations – based on iteration velocity. Finally, the team distributed features into iterations considering priorities, resource demands, and dependencies. The team usually spent some hours on this last step.

4.2. Data collection and results

Seven data sets (Collateral evaluation, Risk assumption, Ukrainian deal flow I–II, Romanian deal flow I–III – respectively R_A to R_C) were selected to make a comparison between the algorithmic method and the manual release scheduling carried out previously at Multilogic. The R_C data set is used to present the concept in the previous example (Figs. 2 and 3a and b). All releases had same project members (6 developers), iteration length (2 weeks), iteration velocity (30 Story point), domain, customer, and development methodology, but they were characterized by different feature counts (FC), iteration counts (IC), buffer per releases (BpR) (for contingencies), and feature size (in working hours) per iteration (FS_i). Table 5 summarizes the variables R_A to R_C collected from the backlog.

To determine the usefulness of our proposed method, we used the historical data as input of the *mksched* algorithm (Algorithm 2). This method made it possible to compare characteristics of the algorithmic approach against the manual one. Computed values (R_A^* to R_C^*) are shown in Table 6 (since FC were the same as in Table 5 they are not shown).

4.3. Analysis

The analysis goal was to compare the manual and the optimized approaches using the same input variables. The following key questions were addressed: **Q1**: What are the staffing requirements over time?; **Q2**: How many iterations do we need per release?; and **Q3**: How buffers for contingencies are allocated?

Table 5
Historical release schedule values (R_A to R_G).

	FC	IC	BpR	FS ₁	FS ₂	FS ₃	FS ₄	FS ₅	$\sum_{i=1}^5 FS_i$
R_A	33	4	3.0	28.0	35.0	24.0	30.0	0.0	117.0
R_B	25	3	4.5	33.0	34.5	18.0	0.0	0.0	85.5
R_C	27	5	12.5	31.5	33.0	23.0	26.0	24.0	137.5
R_D	27	4	3.5	29.5	33.0	27.0	27.0	0.0	116.5
R_E	53	4	−6.5	32.0	34.0	26.5	34.0	0.0	126.5
R_F	26	4	0.0	36.0	27.0	26.0	31.0	0.0	120.0
R_G	53	5	−10.0	34.5	35.0	30.0	33.5	27.0	160.0

Table 6
Optimized release plan values (R_A^* to R_G^*).

	IC	FS ₁ [*]	FS ₂ [*]	FS ₃ [*]	FS ₄ [*]	FS ₅ [*]	FS ₆ [*]
R_A^*	4	30.0	30.0	30.0	27.0	0.0	0.0
R_B^*	3	30.0	29.5	26.0	0.0	0.0	0.0
R_C^*	5	30.0	30.0	30.0	29.5	18.0	0.0
R_D^*	4	30.0	30.0	30.0	26.5	0.0	0.0
R_E^*	5	30.0	30.0	30.0	30.0	6.5	0.0
R_F^*	5	30.0	30.0	29.0	26.0	5.0	0.0
R_G^*	6	30.0	30.0	30.0	30.0	30.0	10.0

We constructed $Y_k \triangleq \sum_{j=1}^n w_j x_{kj}$ (i.e. sum of assigned features to iteration) as a *result variable* to answer our questions (Q1–3). First, we carried out Exploratory Data Analysis (EDA) [50] to get an insight into the data sets, then we performed descriptive statistical analysis to quantitatively compare the main properties of the two approaches. The results of the analysis are presented as follows.

4.3.1. Qualitative analysis

The following EDA techniques (called 4P EDA) are simple, efficient, and powerful for the routine testing of underlying assumptions [50]:

1. *run sequence plot* (Y_k versus iteration k)
2. *lag plot* (Y_k versus Y_{k-1})
3. *histogram* (counts versus subgroups of Y)
4. *normal probability plot* (ordered Y versus theoretical ordered Y)

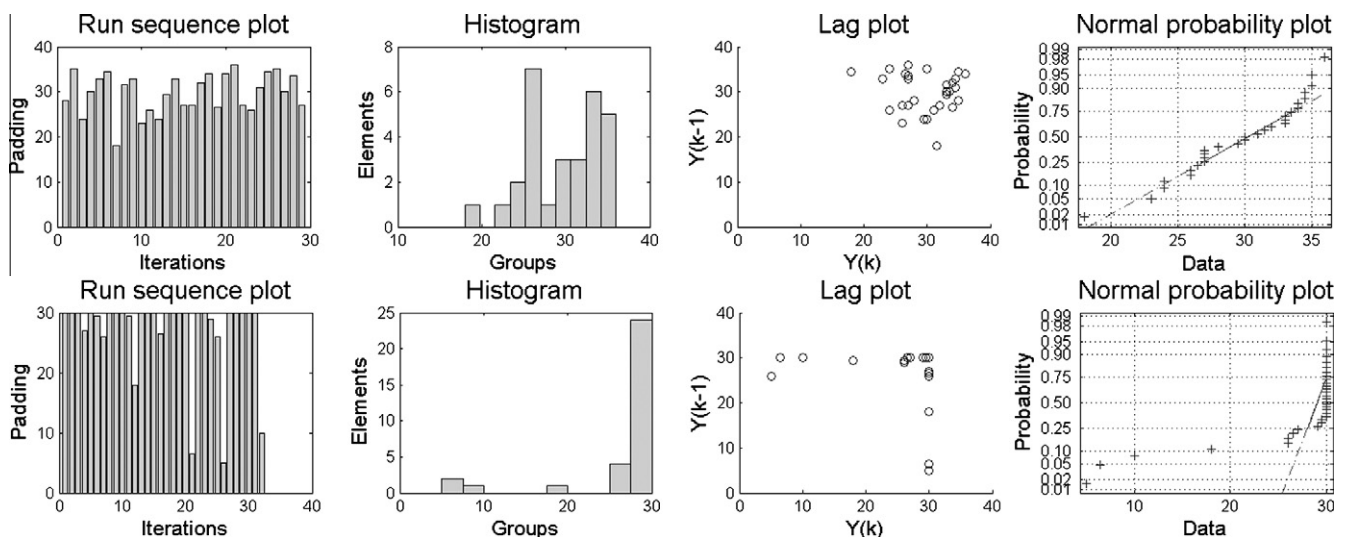


Fig. 4. 4P of historical (upper) and optimized (lower) plans.

The four EDA plots are juxtaposed for a quick look at the characteristics of the data (Fig. 4). The assumptions (A1–4) are addressed by the graphics (Fig. 4):

- A1:** The run sequence plots indicate that the data do not have any significant shifts in location but have significant differences in variation over time.
- A2:** The upper histogram depicts that the data are skewed to the left, there is no significant outliers in the tails, and it is reasonable to assume that the data are from approximately a normal distribution. Contrary, lower one shows asymmetry (skewed to the left heavily), data are more peaked than the normal distribution. Additionally, there is a limit (30) in the data of the algorithmic case that can be explained by the subject of the optimization.
- A3:** The lag plots do not indicate any systematic behavior pattern in the data of the historical case. Though, in the optimized case, there is a dense area at the surrounding of point (30,30) which indicates efficient assignments: considerable amount of iterations are packed with the limit value (30) roughly. The horizontal and vertical lines can be explained by the fact that every release is ended with partial assignments (last iterations contain contingencies).
- A4:** In the upper diagram, the normal probability plot approximately follows straight lines from the 1st to the 3rd quartiles indicating normal distributions. In contrast, the normality assumption is not reasonable in the lower case.

Interpreting the above plots, we can statistically conclude that there is no correlation among the historical data (A3), while it follows approximately a normal distribution (A4), and the optimized approach points out correlation (A3), and yields more smooth release padding and less variance (A1,A2).

These statistical conclusions point out some important differences between the optimized and the historical release planning cases. On the one hand, apart from the last iterations in each release, all iterations were (almost) smoothly and fully padded that can be explained by the effective packing capability of the algorithm which yielded economically well exploited iterations (resources are not underloaded). Additionally, the algorithmic case also avoided resource overloading (exceeding the resource limit) which overload may lead to increasing delivery risk. In contrast, in the manual method some iterations were over- and others were

underloaded (the paddings were considerable smaller or larger than the limit).

4.3.2. Quantitative analysis

The qualitative analysis pointed out an important aspect of the algorithm usage in scope-driven situations (see Section 2.5): applying the algorithm on a given input set, without considering the scheduling parameters, the algorithm may result in ineffective paddings in the last iterations which constitute non-economical resource utilizations. This observation can be explained by the fact that the algorithm had to add one more iteration to the releases R_E^* to R_G^* to avoid resource overload (cf. Tables 6 and 5). (This effect cannot be observed in date-driven cases since it is usually assumed that there are more deliverable features than that can be delivered in a given time period – so all iterations are well padded.) So, it suggests that the algorithm yields an automatic overload protection due to the iteration velocity constraints (c_k), but underload protection requires human intervention by adjusting the input parameters of scheduling.

Actually, in real life situations, the project manager usually iteratively adjusts the parameters of scheduling to deliver the most valued features with as few resources (i.e. cost) as possible. The latter one can be realized with resource underload avoidance. Practically, this endeavor can be implemented by (1) decreasing the scope of the release to get rid of the last ineffective iteration, (2) increasing the scope of the release to fully pad the last iteration, or (3) decreasing the length of the last iteration. To simulate this project manager's endeavor, we also constructed an other optimized case – denoted by R_{A-G}^{**} – by leaving out the last iteration from the release in order to get rid of the ineffective iteration. We consider this case as a more realistic resource utilization than R_{A-G}^* since it avoids resource underload. As scope reduction does not affect answering the Q1 and Q3, we used this case in the comparisons also.

The previous data sets were analyzed with descriptive statistics (D1–3) (Table 7) to point out the quantitative differences between the historical and optimized cases. These statistics (D1–3) not just point out the need of human intervention, but also support the suggested conclusions of the qualitative analysis:

- D1:** despite the iteration velocity was 30, the release schedule – in the historical case – prescribed 36, which resulted in 20% resource overload (see Max column). The previously mentioned fact, namely the algorithm had to add one more iteration to the releases R_E^* to R_G^* in order to avoid resource overload, explains the 5 value in the Min column of the R_{A-G}^* case. Contrary, in the R_{A-G}^{**} case the worst resource utilized iteration (Min column) is economically better than the historical case by $(26 - 18)/30 = 27\%$.
- D2:** relatively large (4 times) skewness (measure of the asymmetry of the data around the sample mean) of the R_{A-G}^* case (histogram in Fig. 4) can be interpreted by the capacity constraints (see (2b)) and the objective (see (2a)). Additionally, comparing the historical case with the R_{A-G}^{**} case it shows an even larger (6 times) skewness, that can be explained by the fact that the algorithm can pack the iterations in even better (cf. minimum values) if the manager considers the size of the development scope.

Table 7
Comparison with descriptive statistics.

	Min	Max	Skewness	Kurtosis
R_{A-G}	18.0	36.0	−0.61	2.79
R_{A-G}^*	5.0	30.0	−2.40	7.28
R_{A-G}^{**}	26.0	30.0	−3.64	15.24

Table 8
Comparison of the historical and the R_{A-G}^{**} paddings.

	$\Delta \geq 20\%$	$20\% > \Delta \geq 10\%$	$10\% > \Delta > 0$	$\Delta = 0$
R_{A-G} (%)	17	59	17	7
R_{A-G}^{**} (%)	0	4	12	84

D3: relatively large kurtosis (measure of how outlier-prone a distribution) of the R_{A-G}^* case (histogram in Fig. 4) can also be explained by the capacity constraints (see (2b)) and the objective (see (2a)) of the optimization. Considering the R_{A-G}^{**} case, the higher kurtosis means less frequent extreme deviations which is economically better also.

Referring back to the Table 5, there were 4 iterations in the historical case where the resources were underloaded more than or equal to 20% ($FS_i \leq 24$), there were 7 iterations where the underloads were more than or equal to 10% but less than 20% ($24 < FS_i \leq 27$), and there were 2 iterations where the underloads were less than 10% but the paddings were not optimal ($27 < FS_i$ and $FS_i \neq 30$). Considering overload, there was 1 iteration in the historical case where the resources were overloaded more than or equal to 20% ($FS_i \geq 36$), there were 10 iterations where the overloads were more than or equal to 10% but less than 20% ($36 > FS_i \geq 33$), and there were 3 iterations where the overloads were less than 10% but the paddings were not optimal ($33 > FS_i$ and $FS_i \neq 30$). Contrary, in the R_{A-G}^{**} case, the count of the underloaded iterations were 0, 1, and 3 in the same intervals respectively, and there was not any overloaded iteration due to the iteration velocity constraint (c_k) of the optimization model. The Table 8 summarizes the distances (Δ) from the maximal value (30) expressed in percentages (how many iterations from the total iterations of the given case fell in the defined intervals).

So, in the historical case, 17%, 59%, 17% of the total 29 iterations were over- or underloaded more than or equal to 20%, more than or equal to 10% but less than 20%, less than 10% respectively. Only 7% of the total iterations were maximal. In contrast, in the R_{A-G}^{**} case, 0%, 4%, 12% of the total 25 iterations were over- or underloaded more than or equal to 20%, more than or equal to 10% but less than 20%, less than 10% respectively. But 84% of the total iterations were maximal. Comparing these values, it can be stated that the algorithmic approach with human intervention could provide a considerably better resource utilization than the manual approach. The better padding of R_{A-G}^{**} cannot be explained by the scope reduction since historical paddings showed normal distribution: we could expect similar padding in the historical case if the scope was smaller (Section 4.3.1).

As a consequence, the staffing requirements (cf. Q1) showed more smooth and fully padded iterations (Fig. 4) in the algorithmic cases (both R_{A-G}^* and R_{A-G}^{**}). It means that the algorithm strives to (1) prevent resource overload – which often yields increasing delivery risks, and (2) underload – which captivates economically badly utilized iterations. It is important to note that, totally padded iterations cannot be achieved in every situation due to the dependencies, and/or the relationship between the size of features and the iteration capacities.

Iteration counts per releases (cf. Q2) of the algorithmic case R_{A-G}^{**} exhibited slight differences: in some situations the algorithm had to add one more iteration to prevent resource overload. Therefore, in real life situations, the algorithm should be used in an iterative manner by altering the schedule parameters – which leads to economically better schedules – cf. R_{A-G}^{**} .

Finally, if we consider the buffer per releases (BpR) that is used for contingencies in practice, we can realize major differences also. In the historical case, they were allocated sporadically: they were

considered in some cases (positive BpR values), and in other cases they were not considered (negative BpR values) – see Table 5. Their allocations in time were also occasional – see Run sequence plot in Fig. 4. The R_{A-G}^* case points out the buffer allocation property of the algorithm: time buffers (cf. Q3) are moved to the end of releases due to the optimality criteria (packing as many items into the iteration as possible). So, if the project manager wants to allocate a time buffer to a given release he/she has to decide on the size of the buffer by determining the deliverable feature set and iteration count, and the algorithm automatically allocate it to the end of the release. (Please note, a simple reordering of the iterations is not a satisfactory solution because the dependencies must be considered.) This characteristic indicates that the contingencies are moved to the end of the release while dependencies are considered by the algorithm, which is more advisable to mitigate risks of delivery slippage [52].

4.4. Computational experiments

Problem solving time of the historical data sets, which can be considered as small-medium-sized problems, took less than one second with our prototypic tool. Therefore, to give some orientation about the performance of the *mksched* algorithm on larger problems, we carried out simulations with the guidance of [61].

To get a more nuanced picture of the algorithm we considered several groups of randomly generated instances of release scheduling to reflect special properties that may have influenced the solution process. In all instances the resource demands (w_j) were uniformly distributed in the data set of {0.5, 1, 2, 3, 5, 8} (reflected the typical resource demands in Story points [13]). The values (p_j) were expressed as a function of the resource demands – yielding typical properties of each group such a way that p_j was chosen randomly from $[w_j - w_j/10, w_j + w_j/10]$ interval. It can be explained by the fact that, typically the value differs from the resource demand by only a few percents, since it is well-known that the return of investment (ROI) is generally proportional to the sum invested within some variations (they are highly correlated) [60]. To enlarge the problem size, we chose total effort in each iteration (c_k) equal to 50 which can be considered as a typical team velocity of a large agile team (cca. 10-people) [13]. We also generated precedence and coupling relations between the items – 30% and 10% of the feature count in a given iteration respectively – analogically to the collected data sets (Section 4.2).

4.4.1. Solving time

The behavior of the algorithm was considered for different problem sizes $n \in \{50, 100, 200, 500\}$, and it was run 30 times to calculate the mean solving time. The size 50 is similar to the historical data sets (Section 4.2), but 500 practically never occurs since it would require to plan 10 iterations ahead for a large agile team (the typical value is 4 [13]). Additionally, the algorithms were run in three cases: (1) without dependencies ($\setminus(P \cup C)$), (2) without couplings ($\setminus C$), and (3) with dependencies ($P \cup C$) in order to get insight into their effects on the solving time. All tests were run on an Intel Pentium 4, 2.2 GHz, 4GB memory, MS Windows XP SP3. The results are presented in Table 9, where I points out how many iterations were needed to compute the solution, and ST denotes

running time (in milliseconds) of the algorithm.

Based on the results in Table 9, we can conclude that: (1) the more realistic case ($ST^{(P \cup C)}$) was acceptable since the computation time was less than 4 s even in case of large instances (500 features); (2) the $\setminus(P \cup C)$ and $\setminus C$ cases were exponential to the item count (roughly $ST \propto n^{\approx 1.1-1.5}$) as it was expected (see the $O(o * n^2)$ asymptotic upper bound in Section 3.5.2); (3) the $\setminus C$ case was harder than the $\setminus(P \cup C)$ case – roughly one order of magnitude, which can be explained by the complexity of precedence constraint handling; and finally (4) the $(P \cup C)$ case was roughly linear to the problem size, which can be interpreted by the fact that constructing feature packages resulted in smaller problems (fewer item counts). Therefore it required less computation power.

As a consequence, the hardest problems are those ones which have many precedences (as precedence negatively affects the performance). Contrary, coupling relations positively influence the performance, since they decrease the number of schedulable items – not considering that extreme cases when there are too many coupling relations and the constructed feature packages exceed the total capacity of the given iteration, so the problem becomes infeasible (see (4b)). This extreme situations may be handled by partitioning the packages into smaller ones by diminishing the count of coupling relations. As a consequence, we can state that, our proposed algorithm could compute schedules very quickly (within some seconds) both on the historical and on the hypothetical agile release instances.

Considering the ‘simple’ binary multiple knapsack case ($\setminus(P \cup C)$), which does not contain our additional constraints (see Section 3.5.2), our algorithm’s solving time was worse by one or two magnitudes than the solving time of the state-of-the-art public knapsack algorithms that can be found in [61] (Chapter 5). Specifically, the proposed algorithm’s solving time was worse by one magnitude than the solving time of the Horowitz–Sahni algorithm. This slowing-down can be explained by two things. On the one hand, our proposed algorithm requires additional iterations and administration to deal with the introduced extensions – particularly the multiple knapsack capabilities and the precedence handling (see Section 3.5.2). The additional iterations, that are emanated from the multiple knapsack capabilities, are manifested in the external loop of the algorithm (see lines 5–68 in Algorithm 2). Actually it is also expressed in the o parameter of the algorithm time complexity (Section 3.5.2). (Please note, the time complexity of the Horowitz–Sahni algorithm and the proposed algorithm is $O(n^2)$ [47] and $O(o * n^2)$ (Section 3.5.2) respectively.) On the other hand, the additional administration can be explained by the precedence handling, namely maintaining ready list and scheduled-list (see lines 6 and 7), of the proposed algorithm.

Revert to the comparison of our algorithm and the state-of-the-art public knapsack algorithms, we have to underline the fact that we implemented our algorithm on the Matlab platform, which executes the code in an interpreted way, therefore, compiled versions (e.g. implemented in C) are expected to be faster with one order of magnitude at least. Even if the worst comes to the worst and our algorithm would not be accelerated by the compilation in that degree, our algorithm is able to handle the introduced constraints of agile release scheduling while it computes the results within an acceptable response time.

4.4.2. Quality of plans

The objective function (see (2a)) points out the maximal cumulative business value delivery endeavor of the ARS optimization model. This model tries to deliver maximal cumulative business value in each iteration: the greatest value in the first iteration, the second greatest one in the second iteration and so on.

Table 9
Simulation results of solving time.

n	$I^{(\setminus(P \cup C))}$	$ST^{(\setminus(P \cup C))}$ (ms)	$I^{(\setminus C)}$	$ST^{(\setminus C)}$ (ms)	$I^{(P \cup C)}$	$ST^{(P \cup C)}$ (ms)
50	234	90	16,545	1742	6443	651
100	1690	208	53,020	5565	6883	711
200	2058	684	72,880	7653	17,090	1812
500	33,164	4413	273,987	29,392	34,232	4116

Table 10
Simulation results of plan quality.

n	c_k	$\backslash(P \cup C)$				$\backslash C$				$(P \cup C)$			
		δ_1 (%)	δ_2 (%)	δ_3 (%)	δ_4 (%)	δ_1 (%)	δ_2 (%)	δ_3 (%)	δ_4 (%)	δ_1 (%)	δ_2 (%)	δ_3 (%)	δ_4 (%)
50	50	0	0	2	98	5	0	0	95	0	1	0	99
	40	0	0	0	100	6	5	4	85	0	3	5	92
	30	0	0	4	96	10	0	8	82	3	0	11	86
100	40	0	0	0	100	0	0	2	98	0	0	0	100
	40	0	0	0	100	2	0	2	96	0	0	2	98
	30	0	0	2	98	3	0	3	94	0	0	2	98
200	50	0	0	0	100	0	0	0	100	0	0	0	100
	40	0	0	0	100	0	0	1	99	0	0	0	100
	30	0	0	0	100	1	0	1	99	0	0	0	100
500	50	0	0	0	100	1	0	0	99	0	0	0	100
	40	0	0	0	100	0	0	0	100	0	0	0	100
	30	0	0	0	100	0	0	0	100	0	0	0	100
$\bar{A}$		0	0	0.7	99.3	2.4	0.4	1.7	95.5	0.2	0.3	1.7	97.8

Let us given a set of deliverable features j ($j \in \mathcal{W} : |\mathcal{W}| = n$) with priorities p_j (reflect the business value for the customer), resource demands w_j , and iterations k with different iteration velocities c_k within a release (see Section 3.4). Assignments of features to iterations depend on the iteration velocity (c_k), the precedence (P), and the coupling (C) constraints of the model while the objective function is considered. In spite of the fact that the algorithm produces a global optimal solution, in Section 4.3.2 (see Table 8) it is revealed that the algorithm could not achieve complete utilization of resources (i.e. fully packed iterations) in every scheduling case owing to the constraints of the model. Thus, in this section, we investigate this distance (Δ) from the maximal value expressed in percentages (how many iterations from the total iterations of the given case fell in the defined intervals) as a function of the adequate scheduling parameters (i.e. n , c_k , C and P). (Please note, w_j and p_j parameters are not considered. As the former one interacts with c_k and the latter one does not play a role in capacity filling.)

Let us define the quality of release plans in terms of the distances (Δ) from the theoretical maximal resource utilization (i.e. from the c_k). This distance characterizes the underload of iterations only, since overloads are avoided by the c_k constraint of the model (see Section 4.3.2).

The previous randomly generated instances of release data were analyzed to measure the quality of the release plans. We investigated the release plans considering the previously defined item counts ($n \in \{50, 100, 200, 500\}$), and the $c_k \in \{30, 40, 50\}$ instances to analyze the influences of different iteration capacities (small, normal, large capacity – respectively) on the quality of the plans. The algorithms were run in three cases as before: (1) without dependencies ($\backslash(P \cup C)$), (2) without couplings ($\backslash C$), and (3) with dependencies ($P \cup C$) in order to get insight into their effects on the quality. The algorithms were run 30 times to calculate the average distances of the defined intervals. The Table 10 summarizes the distances (Δ) in percentages. Please note, to save space in the Table, the previously defined distance intervals (see Table 8) are abbreviated as $\delta_1 \triangleq (\Delta \geq 20\%)$; $\delta_2 \triangleq (20\% > \Delta \geq 10\%)$; $\delta_3 \triangleq (10\% > \Delta > 0)$ and $\delta_4 \triangleq (\Delta = 0)$. Additionally, the average distance in an interval is denoted with $\bar{A}$. Considering these intervals along with the investigated parameters (n , c_k , C and P), we can observe that:

PQ1: looking at the $\bar{A}$ values in the table, one can realize that $\backslash(P \cup C)$ case produced the highest value in the δ_4 column and the lowest values in the δ_{1-3} ones, therefore, it provided the best resource utilization. Comparing the remained cases, the $(P \cup C)$ case is better than the $\backslash C$ case. This order can be explained by the complexity of precedence handling and the

different feature sizes. The $\backslash(P \cup C)$ case did not deal with any dependencies, so it enabled considerably larger search space for the algorithm. (Generally, the larger search space leads to greater variations of packing and often may lead to a better solution.) The $(P \cup C)$ case bothered with precedences and couplings where the latter one constituted fewer precedences and larger item sizes (see Section 4.4.1). Fewer precedences positively affect the finding of better packing due to the larger search space. Whereas, the larger item sizes negatively influence on the packing, since it results in fewer item counts therefore smaller search space. Finally, the $(P \cup C)$ case dealt with the greatest number of precedences that constituted the worst packing.

PQ2: pondering the item counts (n), we can notice that the more items the better packing. The most significant difference can be seen between the $n = 50$ and $n = 500$ instances. In $n = 50$ instances (including $\backslash(P \cup C)$, $\backslash C$ and $(P \cup C)$ cases), the value of the interval δ_4 was between 82% and 100%. While in $n = 500$ instances, the value of the interval δ_4 was between 99% and 100%. This phenomenon can be explained by the fact that greater item counts constitute larger search space.

PQ3: considering the iteration capacity (c_k), we can realize that the capacity positively influences on the packing under a given item count (i.e. $n \leq 100$): the larger the capacity the better the packing. This observation can be explained by two things. Firstly, the larger the capacity the greater the variation of packing due to the greater search space. Secondly, the greater item counts – above a certain level (cca. $n \geq 100$) – can compensate the negative effect of the capacity constraint.

Interpreting the observation **PQ1–PQ3** we can statistically conclude that (1) the number of precedences and couplings (**PQ1**) negatively correlate with the degree of resource utilization – therefore, the degree of release plan quality. On the other hand, both the number of items (**PQ2**) and the extent of iteration capacity (**PQ3**) positively correlate with the degree of plan quality. As a consequence, these parameters should be considered by the project manager during release planning to provide a satisfactory quality level.

One can notice that the $(P \cup C)$ case with $n = 50$ and $c_k = 30$ parameters is similar to the post-mortem optimized case (R_{A-G}^{**} case – see Table 8). We think of this type of problem as a small-medium-sized agile release scheduling problem. Actually, we also tried to carry out simulations of the $(P \cup C)$ case with $n = 50$ and $c_k = 20$ case to simulate small sized problems. But in a number of cases the algorithm met with infeasible scheduling problems due

to the relatively large size of coupled items (i.e. in many cases the schedulable items were greater than the capacity of release). This event very rarely happened at parameter $c_k = 30$ and therefore, in this case, we considered them as outliers. In real life situations, this problem may be tackled with either partitioning of the items or increasing the iteration time period that yields greater capacities. As a consequence, we can conclude that the feature size may influence on the plan quality also: the smaller items the better quality. This observation is particularly important in small-sized agile release scheduling problems.

Comparing all the differently parameterized simulations with the historical case, despite the simulation problems were more complex than the historical one, the algorithmic approach could outperform the manual approach in terms of release plan quality (cf. Section 4.3.2).

5. Discussion and future work

Release planning is an activity concerned with the implementation of the selected features (*what* aspect) in the next version of the software. Complexity of planning arises from the interaction between features by *implicit* and *explicit* constraints. While the previous is given by the scarcity of resources (available developers), the latter one is emerged from different dependencies (cf. Section 3.1). Without dependencies, release planning can be considered as an 0-1 *knapsack problem*, which is NP-hard [47]. With dependencies planning becomes a more complex problem. Though, there are some proposed techniques to release planning, they do not consider the staged-delivery (*when*) characteristics of agile environments – however, this timing factor strongly influences the selection and assignment of features to iterations. In fact, when both *what* and *when* factors are considered, the problem becomes scheduling so we called it – *agile release scheduling*.

In agile environments, all scheduling factors (P1–5 in Section 1.2) are managed informally in planning meetings. Informal approaches work well in smaller projects, however as the size and complexity increase planning becomes a very complex, time consuming process and advocates tool support [23,26]. To address this situation, we proposed a combinatorial optimization model and algorithm as a solution.

First, we identified two release scheduling dependencies (i.e. *coupling* and *precedence*) between features (cf. Section 3.1) that affect implementation sequences. We concluded that five dependencies can be interpreted as release scheduling dependencies from the six types of dependencies identified in [21]. Whereas XOR is heavily used in practice during requirement selection, we ignore it from our solution since release planning and scheduling practically happen after the scope is specified so when the selection among alternatives is previously done. Goal models have been found to be effective for determining the scope of the scheduling [53–55]. They can help to identify variability at the early requirement's phase by capturing alternatives by which stakeholders can achieve their goals.

Secondly, to formulate the agile scheduling model, we identified the main concepts of agile scheduling (cf. Section 3.2). Although, there can be found partial conceptual models in [28,29], but a full-scaled model could not be found. So, we did not only present release but iteration scheduling concepts to provide a global view of agile scheduling. These concepts not only helped to identify the objects and subject of the optimization but with the precise relationships it can also be used in the design of agile release and iteration scheduling applications.

We formulated agile release scheduling as an extension of the binary multiple knapsack optimization problem [47] – considering the previous factors (P1–5). Our proposed solution covers

wide-ranging release scheduling situations with the expression of: (1) date-driven/scope-driven scheduling (cf. Section 2.5), (2) dependencies between features (cf. Section 3.1). We called this as agile release scheduling problem (ARSP). This interpretation made it possible to adapt efficient global optimization algorithms – we utilized the search concept of Horowitz–Sahni's algorithm [47] – to solve ARS problems. The developed algorithm (*mksched*), in terms of (1) staffing requirements (cf. Q1): it showed more smooth and fully padded iterations that mean that the algorithm strives to (1) prevent resource overload – which often yields increasing delivery risks, and prevent resource underload – which captivates economically badly utilized iterations; (2) iteration counts per releases (cf. Q2): it pointed out that to avoid resource underload it should be used in an iterative manner by the project manager; finally, (3) release buffers (cf. Q3): it supports lower level risk of delivery slippage [52]. Moreover, addressing the date-driven/scope-driven scheduling questions (cf. Section 2.5) – contrary to the common date-driven approach (e.g. in [35]) – and the easy and fast computation – even on large problems (Section 4.4) – allows (1) generating alternative schedules by utilizing what-if analysis to tailor the best schedule for the specific project context and (2) considering the stakeholders' feedbacks by altering constraints and priorities. Both lead to more easily adaptable schedules and more informed and established decisions by selecting the best schedule from the alternatives. The developed algorithm can be downloaded from the author's website.

However, several optimization algorithms are publicly available (see in [47,61]) and they can solve common knapsack models very efficiently, the previously presented ARS optimization model requires a custom-designed algorithm (an extended version of the Horowitz–Sahni's algorithm [47]) due to the followings:

1. *Scheduling capability*: Common knapsack models can calculate the next releasable items in contrast with our proposed solution, which can handle many (theoretically any) iterations immediately – due to the demand of staged-delivery approach of agile environments. As a consequence, our (ARS model and) *mksched* algorithm not just determines the *what-aspect*, but the *when-aspect* also (in which iteration the selected items will be delivered). Therefore, we named this model as *agile release scheduling*.
2. *Soft precedence*: Multi-knapsack algorithms generally do not handle any precedence, so we had to complement it. This extension can handle precedences internally in each iteration: even if there is precedence relation between two items both may be put into a given iteration – as far as there is free capacity in the iteration. Consequently, this solution often leads to a far better solution than the usual precedence constrained solutions (e.g. in [35] and in [28]) which ones deal with precedences externally (i.e. if an item is put into iteration k then its successor must be put into iteration $>k$).

The presented ARS optimization model can be also implemented by several optimization tools (such as CPLEX [56], AIMMS [57]), and they may provide better solving times (but not better quality since our algorithm finds the global optimum). However, there may be some practical drawbacks using these packages: (1) optimization problem modeling with these tools requires special mathematical modeling and/or ILP tool knowledge in comparison to implementing the presented algorithm in any popular programming language, and the lack of this knowledge is often a barrier to put the proposed models into practice; (2) integrating the callable tool libraries may be difficult (e.g. CPLEX connector is not available); finally, (3) buying a tool (e.g. CPLEX, AIMMS) that is made by professionals in order to solve just one problem is rather expensive (typically $n * \$1000$).

We can look at the proposed solution as an extension to the readily available agile planning tools (such as Rally [11] or XPlanner [12]) which helps collecting the planning data (features, required effort, team velocity, etc.), but they support only manual release planning. (At Multilogic the planning data were collected through an MS Sharepoint-based web site [62].) Therefore, with this extension, we believe that one can produce agile release schedule easily based on the collected data. We also expect that our proposed method requires a little more effort (some minutes) in set up time in expressing dependencies between deliverable features, but it produces optimal schedule within seconds – in contrast to the manual scheduling which required some hours (see Section 4.1). But actually, the major difference is not the required effort but the higher quality of the schedule: the optimized approach provides a higher quality schedule (i.e. avoiding under- and overload), produces better resource utilization (see Section 4.4.2), and makes it possible to re-schedule it any time within seconds in order to support what-if analysis or to adapt the schedule to a changed situation.

More comprehensive empirical data is necessary to evaluate the quality of our proposed agile release scheduling method, but the results suggest that the project manager with this algorithm could realize less risky (not overloaded and better time buffer allocated) and more economical (rarely slightly underloaded) release schedules by altering the input parameters of the algorithm. The presented conceptual model of agile release and iteration scheduling (cf. Section 3.2) points out that this approach can be complemented with an iteration-level scheduling i.e. allocating `TechnicalTasks` to `Developers`. Additionally, since the presented solution pertains to single team release scheduling only, it can also be extended with multi-team scheduling capabilities to support distributed development situations (such as Isolated Scrums or Fully Distributed Scrum[58,59]). A complementary iteration planning method and distributed agile release planning extension can be found in [29] and in [30], and they fit well to the presented agile release scheduling approach. An integrated solution may provide not just one agile team's but many teams' scheduling, and not just at the release-level but at iteration-level also.

6. Conclusions

The three out of the four most important barriers of agile adoptions are the lack of upfront planning, the loss of management control, and the lack of predictability [1]. These barriers are closely connected with the present practice of agile planning. These critics initiated our endeavor to diminish these barriers. First, we provided a proposed conceptual model of agile planning that defines a common language to avoid the different interpretations that are used in the field. This model not only helps to identify and define the main notions but with its precise relationships it can also be used in the design of agile planning applications.

The present practice of agile release planning is unsatisfactory due to lack of solid theoretical basis and existence of a number of competing practices and fads. Therefore, as a second aim, we proposed a theoretically sound method for agile release planning. In agile environments, scheduling features into the upcoming delivery stages is a complex, informal and mainly manual process. Up to our best knowledge, the proposed staged-delivery global optimized model formulation is novel in the field. Although, there are some tenets to manual scheduling [3,13] algorithmic solutions could not be found. The proposal gives the main parameters of the typical agile planning space (such as objectives, constraints) and presents an optimization model that can be realized by optimization tools or by implementing our suggested custom-made algorithm. To evaluate our model simulations were carried out seven

real life and several generated data sets that demonstrated that the method could easily cope with the previously manually managed scheduling factors (P1–5) besides providing optimized schedules. Additionally, this approach provides more informed and established decisions with application of what-if analysis (re-scheduling the release by altering its parameters), and mitigates risks by providing more smooth feature allocation and moving buffers to the end of releases. We believe the results are even more impressive in more complex (more constraints, features, etc.) situations.

With the results of experiments on real life and generated data sets indicate that our approach can provide practical value as a decision support tool for product and development project managers. We believe these results may help to diminish the previously mentioned barriers of agile adoption.

Acknowledgements

The author would like to thank the associate editor and the anonymous referees for their valuable comments and suggestions. Discussion with colleagues at Multilogic also helped to provide a more readable content.

References

- [1] VersionOne Corp. 4rd Annual Survey: 2009, The State of Agile Development. <<http://www.versionone.com/agilesurvey/>> (accessed on 01.11.10).
- [2] Manifesto for agile software development. <<http://www.agilemanifesto.org>> (accessed on 27.02.10).
- [3] T. Dybå, T. Dingsøyr, Empirical studies of agile software development: a systematic review, *Inform. Softw. Technol.* 50 (2008) 833–859.
- [4] S.W. Ambler, Survey says: agile works in practice, *Dr. Dobbs J.* 31 (2006) 62–64.
- [5] L. Layman, L. Williams, L. Cunningham, Motivations and measurements in an agile case study, *J. Syst. Architect.* 52 (2006) 654–667.
- [6] L. Layman, L. Williams, L. Cunningham, Exploring extreme programming in context: an industrial case study, in: *ADC '04: Proc. of the Agile Development Conf.*, 2004, pp. 32–41.
- [7] K. Beck, C. Andres, *Extreme Programming Explained: Embrace Change*, second ed., Addison-Wesley Professional, 2004.
- [8] S.R. Palmer, M. Felsing, *A Practical Guide to Feature-Driven Development*, Pearson Education, 2001.
- [9] T. Chow, D.-B. Cao, A survey study of critical success factors in agile software projects, *J. Syst. Softw.* 81 (2008) 961–971.
- [10] P. Stevens, M. Dubakov, Agile tools: the good, the bad, the ugly. *Agile J.* (2008) (white paper).
- [11] Rally Software Dev. Corp. Rally homepage. <<http://www.rallydev.com/>> (accessed on 04.09.10).
- [12] Codehaus Corp. Xplanner homepage. <<http://xplanner.codehaus.org/>> (accessed on 04.09.10).
- [13] M. Cohn, *Agile Estimating and Planning*, Prentice Hall, NJ, USA, 2005.
- [14] J. Karlsson, Software requirements prioritizing, in: *ICRE '96: Proc. of the 2nd Int. Conf. on Requirements Eng.*, IEEE, Washington, USA, 1996, p. 110.
- [15] D. Leffingwell, D. Widrig, *Managing Software Requirements: A Unified Approach*, Addison-Wesley Professional, 1999.
- [16] K.E. Wiegiers, *Software Requirements*, second ed., Microsoft Press, 2003.
- [17] T.L. Saaty, *The Analytic Hierarchy Process*, Planning, Priority Setting, Resource Allocation, McGraw-Hill, New York, 1980.
- [18] A. Perini, F. Ricca, A. Susi, Tool-supported requirements prioritization: comparing the ahp and cbrank methods, *Inform. Softw. Technol.* 51 (2009) 1021–1032.
- [19] J. Karlsson, K. Ryan, A cost-value approach for prioritizing requirements, *IEEE Software* 14 (1997) 67–74.
- [20] H.-W. Jung, Optimizing value and cost in requirements analysis, *IEEE Software* 15 (1998) 74–78.
- [21] P. Carlshamre, K. Sandahl, M. Lindvall, B. Regnell, J.N. Dag, An industrial survey of requirements interdependencies in software product release planning, in: *RE '01: Proc. of the Fifth IEEE Int. Symp. on Requirements Engineering*, IEEE Computer Society, Washington, USA, 2001, pp. 84–93.
- [22] P. Carlshamre, Release planning in market-driven software product development: provoking an understanding, *Req. Eng.* 7 (2002) 139–151.
- [23] G. Ruhe, M.O. Saliu, The art and science of software release planning, *IEEE Softw.* 22 (2005) 47–53.
- [24] M. van den Akker, S. Brinkkemper, G. Diepen, J. Versendaal, Software product release planning through optimization and what-if analysis, *Inform. Softw. Technol.* 50 (2008) 101–111.

- [25] C. Li, J.M. van den Akker, S. Brinkkemper, G. Diepen, Integrated requirement selection and scheduling for the release planning of a software product, in: REFSQ'07: Proc. of Int. Conf. on Req. Eng., Springer, 2007, pp. 93–108.
- [26] A. Aurum, C. Wohlin, The fundamental nature of requirements engineering activities as a decision-making process, *Inform. Softw. Technol.* 45 (2003) 945–954.
- [27] A. Szóke, A proposed method for release planning from use case-based requirements, in: Proc. of the 34th Euromicro Conf., Euromicro SEAA, Parma, Italy, 2008, pp. 449–456.
- [28] A. Szóke, Bin-packing-based planning of agile releases, in: ENASE 2008/2009 Revised Selected Papers, CCIS, Springer, 2010, pp. 133–146.
- [29] A. Szóke, Decision support for iteration scheduling in agile environments, in: PROFES '09: Proc. of the 10th Int. Conf. on Product Focused Software Process Improvement, Oulu, Finland, 2009, pp. 156–170.
- [30] A. Szóke, Optimized feature distribution in distributed agile environments, in: PROFES '10: Proc. of the 11th Int. Conf. on Product Focused Software Process Improvement, Limerick, Ireland, 2010, pp. 62–76.
- [31] D. Greer, G. Ruhe, Software release planning: an evolutionary and iterative approach, *Inform. Softw. Technol.* 46 (2004) 243–253.
- [32] G. Ruhe, A. Eberlein, D. Pfahl, Quantitative winwin: a new method for decision support in requirements negotiation, in: Proc. of the 14th Int. Conf. on Software Eng. and Knowledge Eng., New York, USA, 2002, pp. 159–166.
- [33] N. Fenton, W. Marsh, M. Neil, P. Cates, S. Forey, M. Taylor, Making resource decisions for software projects, in: ICSE '04: Proc. of the 26th Int. Conf. on Soft. Eng., IEEE, Washington, USA, 2004, pp. 397–406.
- [34] C.K. Chang, M.J. Christensen, T. Zhang, Genetic Algorithms for Project Management, vol. 11, J.C. Baltzer AG, NJ, USA, 2001.
- [35] An Ngo-The, G. Ruhe, Optimized resource allocation for software release planning, *IEEE Trans. Softw. Eng.* 35 (2009) 109–123.
- [36] P. Abrahamsson, O. Salo, J. Ronkainen, J. Warsta, Agile software development methods – review and analysis, Technical Report 478, VTT, 2002.
- [37] Z. Racheva, M. Daneva, K. Sikkil, Value creation by agile processes: methodology or mystery? in: Proc. of the 10th Int. Conf. on Product Focused Software Process Improvement, Oulu, Finland, 2009, pp. 141–155.
- [38] K. Schwaber, M. Beedle, Agile Software Development with Scrum, Prentice Hall PTR, Upper Saddle River, NJ, USA, 2001.
- [39] N.B. Moe, T. Dingsøyr, T. Dybå, A teamwork model for understanding an agile team: a case study of a scrum project, *Inform. Softw. Technol.* 52 (2009) 480–491.
- [40] Object Management Group, Unified modeling language, version 2.0. UML 2.0. <<http://www.uml.org>> (accessed on 27.02.08).
- [41] B. Nuseibeh, S. Easterbrook, Requirements engineering: a roadmap, in: ICSE – Future of SE Track, 2000, pp. 35–46.
- [42] B. Anda, H. Dreiem, Dag I.K. Sjøberg, M. Jørgensen, Estimating software development effort based on use cases – experiences from industry, in: 4th Int. Conf. on the UML, LNCS, Springer, 2001, pp. 487–502.
- [43] M. Hericko, A. Zivkovic, The size and effort estimates in iterative development, *Inform. Softw. Technol.* 50 (2008) 772–781.
- [44] N.E. Fenton, S.L. Pfleeger, Software Metrics: A Rigorous and Practical Approach, Revised, Course Technology, February 1998.
- [45] Crytal Reports, Crytal reports homepage. <<http://www.crytalreports.com>> (accessed on 27.02.10).
- [46] S. Hartmann, Packing problems and project scheduling models: an integrating perspective, *J. Operat. Res. Soc.* 51 (2000) 1083–1092.
- [47] S. Martello, P. Toth, Knapsack Problems: Algorithms and Computer Implementations, John Wiley & Sons, Inc., New York, NY, USA, 1990.
- [48] R. Diestel, Graph Theory, Springer-Verlag, Heidelberg, 2005.
- [49] A.K. Hartmann, A Practical Guide To Computer Simulation, World Scientific Publishing Co., Inc., River Edge, NJ, USA, 2009.
- [50] W.L. Martinez, Exploratory Data Analysis with MATLAB, Chapman & Hall/CRC, 2004.
- [51] Multilogic Ltd. Multilogic homepage. <<http://www.multilogic.hu>> (accessed on 27.02.10).
- [52] O.I. Tükel, W.O. Rom, S.D. Eksioğlu, An investigation of buffer sizing techniques in critical chain scheduling, *Eur. J. Operat. Res.* 172 (2006) 401–416.
- [53] A. Dardenne, A. van Lamsweerde, S. Fickas, Goal-directed requirements acquisition, *Sci. Comput. Program.* 20 (1993) 3–50.
- [54] J. Castro, M. Kolp, J. Mylopoulos, Towards requirements-driven information systems engineering: the Tropos project, *Inform. Syst.* 27 (2002) 365–389.
- [55] S. Liaskos, A. Lapouchian, Y. Yu, E. Yu, J. Mylopoulos, On goal-based variability acquisition and analysis, in: RE '06: Proc. of the 14th IEEE Int. Symp. on Req. Eng., IEEE, Washington, USA, 2006, pp. 76–85.
- [56] ILOG Corporation, Cplex homepage. <<http://www.ilog.com>> (accessed on 27.02.10).
- [57] Paragon Decision Tech. Inc., Aimms homepage. <<http://www.aimms.com>> (accessed on 27.02.10).
- [58] S. Lee, H.-S. Yong, Distributed agile: project management in a global environment, *Emp. Softw. Eng.* 15 (2010) 204–217.
- [59] J. Sutherland, G. Schoonheim, E. Rustenburg, M. Rijk, Fully distributed scrum: the secret sauce for hyperproductive offshored development teams, in: AGILE, Toronto, Canada, 2008, pp. 339–344.
- [60] P. Jalote, Software Project Management in Practice, Addison-Wesley Longman Publishing Co., 2002.
- [61] H. Kellerer, U. Pferschy, D. Pisinger, Knapsack Problems, Springer, 2004.
- [62] Microsoft Corp. MS SharePoint site. <<http://www.microsoft.com/sharepoint/>> (accessed on 27.02.10).